HO CHI MINH NATIONAL UNIVERSITY
UNIVERSITY OF SCIENCE

PROJECT REPORT

FUNDAMENTALS OF AI (CSC14003) - 24C11

# FIRST-ORDER LOGIC & CSPs IN FUTOSHIKI

HO CHI MINH CITY – 2025

**HO CHI MINH NATIONAL UNIVERSITY**

**UNIVERSITY OF SCIENCE**

**PROJECT REPORT**

# INTRO TO AI - 24C11

# FUTOSHIKI

| | |
|---|---|
| **INSTRUCTOR:** | TEACHER LÊ NHỰT NAM |
| **GROUP:** | LVH |

| No. | Full name | Student ID |
|---|---|---|
| **1** | Trần Anh Kiệt | 24127287 |
| **2** | Nguyễn Tấn Hiệu | 24127373 |
| **3** | Nguyễn Anh Khôi | 24127430 |
| **4** | Nguyễn Đức Quý | 24127526 |

HO CHI MINH CITY - 2025

# WORK ASSIGNMENT

| No. | Full name | Student ID | Assigned tasks | Completion |
|---|---|---|---|---|
| 1 | Trần Anh Kiệt | 24127287 | Implement the A* solver, document the A* sections, benchmark the algorithms, prepare charts, comparisons, and record the demo. | 100% |
| 2 | Nguyễn Tấn Hiệu | 24127373 | Implement backward chaining solvers and prolog interpreter, prepare examples, benchmark the algorithms, document the BC sections. | 100% |
| 3 | Nguyễn Anh Khôi | 24127430 | Formalize FOL axioms, derive CNF rules, set up the Github repo and write the FOL/SAT sections of the report. | 100% |
| 4 | Nguyễn Đức Quý | 24127526 | Implement forward chaining and backtracking solvers, document the FC/BT sections, integrate project components, review formatting and final experimental analysis | 100% |

# PROJECT LINKS

**Project Repository and Demonstration Resources**

**GitHub Repository:** github.com/akhoi-ng/Futoshiki-FOL-Inference-Algorithms

**Demo Video:** youtube.com/watch?v=vVEt-v7rCTY

**Google Drive:** https://drive.google.com/drive/folders/1Xa9qbicynhJuXYXjH3t3VjOoq5QZJaTD?usp=sharing

# Contents


**1 FOL** **6**

**1.1 Introduction** . . . . . **6**

**1.2 FOL Vocabulary** . . . . . **6**

1.2.1 Sorts and Constants . . . . . 6

1.2.2 Predicate Symbols . . . . . 6

**1.3 Complete FOL Axioms** . . . . . **7**

**1.4 CNF Conversion: Step-by-Step Derivations** . . . . . **10**

1.4.1 Derivation 1: Axiom A1 (At-Least-One) . . . . . 11

1.4.2 Derivation 2: Axiom A2 (At-Most-One) . . . . . 12

1.4.3 Derivation 3: Axiom A3 (Row Uniqueness) . . . . . 13

1.4.4 Derivation 4: Axiom A4 (Horizontal Less-Than) . . . . . 14

**1.5 Grounding the Knowledge Base** . . . . . **14**

1.5.1 Variable Encoding . . . . . 14

1.5.2 Clause Count Analysis . . . . . 15

**1.6 Implementation** . . . . . **15**

**2 Backward Chaining** **16**

**2.1 From the FOL specification to the executable model** . . . . . **16**

**2.2 Knowledge base and Horn clauses actually loaded by the solver** . . . **17**

**2.3 Core SLD-resolution engine: unification, substitution, and search order** **18**

**2.4 Where the actual Futoshiki reasoning happens: MRV, domain filtering, and forward checking** . . . . . **20**

**2.5 Code-faithful pseudocode of the implemented solver** . . . . . **21**

**2.6 Step-by-step simulation on a $4 \times 4$ puzzle** . . . . . **23**

**2.7 Tested backward-chaining outputs on representative instances** . . . . **29**

**2.8 Experimental evidence from the benchmark data** . . . . . **31**

**2.9 Advantages and limitations of this specific backward-chaining design** **32**

**3 Forward Chaining** **34**

**3.1 Theoretical Foundation and Definitions** . . . . . **34**

3.1.1 Key Concepts . . . . . 34

3.1.2 Data-Driven vs. Goal-Driven Inference . . . . . 34

**3.2 Application to the Futoshiki Problem** . . . . . **34**

3.2.1 Knowledge Base Structure . . . . . 35

**3.3 Inference Rules and Their FOL Basis** . . . . . **35**

3.3.1 Rule 1: Singleton Domain (from A1 + A2) . . . . . 36

3.3.2 Rule 2 and 3: Row and Column Uniqueness (A3, A6) . . . . . 36

3.3.3 Rule 4: Hidden Single (A3 + A6 combined) . . . . . 36

3.3.4 Rule 5: Queue-Based Inequality Propagation (A4, A7, A8, A9) . . . . 37

**3.4 Algorithm Structure (Pseudocode)** . . . . . **37**

3.4.1 The Main Loop . . . . . 37

3.4.2 Propagation Detail . . . . . 38

3.4.3 Complexity Estimate . . . 39
**3.5 Backtracking Fallback . . . 39**
**3.6 Step-by-Step Simulation on a $4 \times 4$ Puzzle . . . 39**
**3.7 Experimental Results . . . 42**
**3.8 Advantages and Limitations . . . 42**
**4 Backtracking with Forward Checking 44**
**4.1 Theoretical Foundation and Definitions . . . 44**
4.1.1 Key Concepts . . . 44
4.1.2 Forward Checking (FC-1): Look-Ahead Pruning . . . 44
4.1.3 Complexity . . . 44
**4.2 Application to the Futoshiki Problem (CSP Formulation) . . . 45**
**4.3 Enhanced Bound Pruning in `compute_domain` . . . 45**
**4.4 Algorithm Structure (Pseudocode) . . . 47**
4.4.1 Main Structure . . . 47
4.4.2 Forward Checking . . . 47
4.4.3 MRV Heuristic . . . 48
**4.5 Step-by-Step Simulation on a $4 \times 4$ Puzzle . . . 48**
**4.6 How BT Connects to FC . . . 50**
**4.7 Experimental Results . . . 50**
**4.8 Advantages and Limitations . . . 51**
**5 A* Search Algorithm 53**
**5.1 Theoretical Foundation and Definitions . . . 53**
**5.2 State Space Formulation . . . 53**
**5.3 Heuristics Design and Analysis . . . 53**
**5.4 Step-by-Step Simulation on a Small Puzzle ($4 \times 4$) . . . 54**
5.4.1 Algorithm Structure (Pseudocode) . . . 54
5.4.2 Illustrative Example: Simulation With Heuristic 3 (AC-3 / Arc Consistency) . . . 57
**5.5 Advantages and Disadvantages . . . 61**
**6 Experimental Results 62**
**6.1 Experimental Environment & Measurement Methodology . . . 62**
**6.2 Detailed Results across 12 Test Cases . . . 62**
**6.3 Visual Charts and Evaluation . . . 64**
**7 Comparative Analysis 68**
**7.1 Runtime Comparison . . . 68**
**7.2 Memory and Search Effort . . . 68**
**7.3 Why Each Algorithm Behaves the Way It Does . . . 69**
**7.4 Does Forward Chaining Actually Help Backtracking? . . . 70**
**7.5 When to Use Which Algorithm . . . 71**
**7.6 Summary . . . 71**
**8 References 72**

# 1 FOL

## 1.1 Introduction

Futoshiki (meaning "inequality" in Japanese) is a logic-based number-placement puzzle played on an $N \times N$ grid. The solver must fill in numbers from 1 to $N$ such that:

1. Every row contains each number from 1 to $N$ exactly once.
2. Every column contains each number from 1 to $N$ exactly once.
3. All inequality constraints between adjacent cells are satisfied.
4. Given cells retain their values.

We formalizes the Futoshiki puzzle using **First-Order Logic (FOL)** with a complete set of axioms that encode all puzzle rules, derives the **Conjunctive Normal Form (CNF)** of several axioms step by step (including Skolemization and clausal form), and explains how these axioms are implemented in the project codebase.

## 1.2 FOL Vocabulary

### 1.2.1 Sorts and Constants

We use a single sort INDEX with domain $\{1, 2, \ldots, N\}$ to represent row indices, column indices, and cell values. For a concrete $4 \times 4$ puzzle, the domain is $\{1, 2, 3, 4\}$.

### 1.2.2 Predicate Symbols

The following shows the vocabulary of our FOL formalization:

| **Predicate** | **Arity** | **Intended Meaning** |
|---|---|---|
| $\mathrm{Val}(i, j, v)$ | 3 | Cell $(i, j)$ is assigned value $v$ |
| $\mathrm{Given}(i, j, v)$ | 3 | Cell $(i, j)$ has the pre-filled clue value $v$ |
| $\mathrm{LessH}(i, j)$ | 2 | There is a "<" constraint between $(i, j)$ and $(i, j{+}1)$ |
| $\mathrm{GreaterH}(i, j)$ | 2 | There is a ">" constraint between $(i, j)$ and $(i, j{+}1)$ |
| $\mathrm{LessV}(i, j)$ | 2 | There is a "<" constraint between $(i, j)$ and $(i{+}1, j)$ |
| $\mathrm{GreaterV}(i, j)$ | 2 | There is a ">" constraint between $(i, j)$ and $(i{+}1, j)$ |
| $\mathrm{Less}(v_1, v_2)$ | 2 | Integer $v_1 < v_2$ (background arithmetic) |

Table 2: Predicate symbols used in the Futoshiki FOL formalization.

In our codebase, indices are **0-based**, while the FOL notation in this report uses **1-based** indexing (from 1 to $N$) to match the project specification. The mapping in code is:

$$\texttt{var_id}(r, c, v, N) = r \cdot N^2 + c \cdot N + (v - 1) + 1$$

This function is implemented in `cnf_generator.py` and provides a connection between $\text{Val}(r, c, v)$ atoms and IDs for SAT encoding.

## 1.3 Complete FOL Axioms

We present the full set of nine axioms (A1–A9) that consists of all Futoshiki rules. These axioms with the instance facts (Given, LessH, GreaterH, LessV, GreaterV, and Less), form the **knowledge base (KB)** for inference.

**Axiom A1 — Every cell has at least one value**

**Axiom 1** (Existence / At-Least-One)**.** *Every cell in the grid must have at least one value from* $\{1, \ldots, N\}$ *(make sure that there are no empty cells):*

$$\forall i\ \forall j\ \exists v\ \ \mathit{Val}(i, j, v)$$

**Codebase reference:** Implemented in `cnf_generator.py`, function `generate_at_least_one(N)`. For each cell $(r, c)$, we generate a single clause contains all possible value literals (make sure that there are no empty cells):

$$\Big[\text{Val}(r, c, 1) \vee \text{Val}(r, c, 2) \vee \cdots \vee \text{Val}(r, c, N)\Big]$$

It produces $N^2$ clauses.

**Axiom A2 — Every cell has at most one value**

**Axiom 2** (Uniqueness / At-Most-One)**.** *No cell can be assigned two different values at the same time:*

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \Big(\mathit{Val}(i, j, v_1) \wedge \mathit{Val}(i, j, v_2)\Big) \Rightarrow v_1 = v_2$$

*Similarly, for any* $v_1 \neq v_2$*:*

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \Big(v_1 \neq v_2\Big) \Rightarrow \Big(\neg\, \mathit{Val}(i, j, v_1) \vee \neg\, \mathit{Val}(i, j, v_2)\Big)$$

**Codebase reference:** Implemented in `generate_at_most_one(N)`. For each cell, it goes through all pairs $(v_1, v_2)$ with $v_1 < v_2$ using `combinations`, generating binary clauses $[\neg\text{Val}(i, j, v_1),\ \neg\text{Val}(i, j, v_2)]$. This produces $N^2 \times \binom{N}{2}$ clauses, based on the logic that a cell cannot simultaneously take both values $v_1$ and $v_2$.

### Axiom A3 — Row uniqueness

**Axiom 3** (Row Uniqueness)**.** *Each row only contains a value from* $\{1, \ldots, N\}$ *once (no duplicate values on the same row):*

$$\forall i\ \forall j_1\ \forall j_2\ \forall v \quad \Big(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2\Big) \Rightarrow \bot$$

*Similarly:*

$$\forall i\ \forall v\ \forall j_1\ \forall j_2 \quad \Big(j_1 \neq j_2\Big) \Rightarrow \Big(\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v)\Big)$$

**Codebase reference:** Implemented in `generate_row_uniqueness(N)`. For each row $i$ and each value $v$, it goes through all pairs of different columns $(j_1, j_2)$ with $j_1 < j_2$, producing $N \times N \times \binom{N}{2}$ binary clauses. Combined with A1, this makes sure each value appears *exactly once* per row (i.e., In range $\{1, \ldots, N\}$ each row is a permutation) (a value $v$ can not be both on cell $(r_1, c_1)$ and $(r_1, c_2)$).

### Axiom A6 — Column uniqueness

**Axiom 4** (Column Uniqueness)**.** *Each column only contains a value from* $\{1, \ldots, N\}$ *once (no duplicate values on the same column) (same as axiom A3 but this is for column):*

$$\forall j\ \forall i_1\ \forall i_2\ \forall v \quad \Big(Val(i_1, j, v) \wedge Val(i_2, j, v) \wedge i_1 \neq i_2\Big) \Rightarrow \bot$$

*Similarly:*

$$\forall j\ \forall v\ \forall i_1\ \forall i_2 \quad \Big(i_1 \neq i_2\Big) \Rightarrow \Big(\neg Val(i_1, j, v) \vee \neg Val(i_2, j, v)\Big)$$

**Codebase reference:** Implemented in `generate_col_uniqueness(N)`. Same as A3, generating $N \times N \times \binom{N}{2}$ binary clauses for column constraints. (i.e., In range $\{1, \ldots, N\}$ each column is a permutation) (same logic as A3).

### Axiom A4 — Horizontal less-than constraint

**Axiom 5** (Horizontal Less-Than)**.** *If there is a "<" constraint between* $(i, j)$ *and* $(i, j{+}1)$*, then the value at* $(i, j)$ *must be less than the value at* $(i, j{+}1)$ *(if a 2 cells on a row has a less-than constraint between them, the cell on the left must be less than the cell on the right):*

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \Big(LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j{+}1, v_2)\Big) \Rightarrow Less(v_1, v_2)$$

**Codebase reference:** Implemented in `generate_horizontal_less(N, puzzle)`. For each position $(r, c)$ where `puzzle.h_con[r][c] == 1` (LessH), it generates clauses removing all pairs that has $(v_1, v_2)$ $v_1 \geq v_2$ (instead of the rule cell $(r, c)$ must be smaller than the cell $(r, c+1)$, if a cell has value $v_1 \geq v_2$ we ban them from putting $v_1$ into cell $(r, c)$ and $v_2$ into cell $(r, c+1)$):

$$[\neg\text{Val}(r, c, v_1),\ \neg\text{Val}(r, c{+}1, v_2)] \quad \text{for all } v_1 \geq v_2$$

**Axiom A7 — Horizontal greater-than constraint**

**Axiom 6** (Horizontal Greater-Than). *If there is a "$>$" constraint between $(i, j)$ and $(i, j{+}1)$, then the value at $(i, j)$ must be strictly greater than the value at $(i, j{+}1)$ (if a 2 cells on a row has a greater-than constraint between them, the cell on the left must be greater than the cell on the right):*

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \Big(\mathit{GreaterH}(i, j) \wedge \mathit{Val}(i, j, v_1) \wedge \mathit{Val}(i, j{+}1, v_2)\Big) \Rightarrow \mathit{Less}(v_2, v_1)$$

**Codebase reference:** Implemented in `generate_horizontal_greater(N, puzzle)`. For each GreaterH position, it removes all pairs that has $(v_1, v_2)$ $v_1 \leq v_2$ (same as A4 but for horizontal):

$$[\neg\mathrm{Val}(r, c, v_1),\ \neg\mathrm{Val}(r, c{+}1, v_2)] \quad \text{for all } v_1 \leq v_2$$

**Axiom A8 — Vertical less-than constraint**

**Axiom 7** (Vertical Less-Than). *If there is a "$<$" constraint between $(i, j)$ and $(i{+}1, j)$, then (if a 2 cells on a column has a less-than constraint between them, the cell above must be less than the cell below):*

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \Big(\mathit{LessV}(i, j) \wedge \mathit{Val}(i, j, v_1) \wedge \mathit{Val}(i{+}1, j, v_2)\Big) \Rightarrow \mathit{Less}(v_1, v_2)$$

**Codebase reference:** Implemented in `generate_vertical_less(N, puzzle)`. Same as horizontal but this time for vertical.

**Axiom A9 — Vertical greater-than constraint**

**Axiom 8** (Vertical Greater-Than). *If there is a "$>$" constraint between $(i, j)$ and $(i{+}1, j)$, then (if a 2 cells on a column has a greater-than constraint between them, the cell above must be greater than the cell below):*

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \Big(\mathit{GreaterV}(i, j) \wedge \mathit{Val}(i, j, v_1) \wedge \mathit{Val}(i{+}1, j, v_2)\Big) \Rightarrow \mathit{Less}(v_2, v_1)$$

**Codebase reference:** Implemented in `generate_vertical_greater(N, puzzle)`. Same as horizontal but this time for vertical.

**Axiom A5 — Given clues are enforced**

**Axiom 9** (Given Clues)**.** *Pre-filled cells must retain their assigned values (must not change cells with values that are already there):*

$$\forall i\ \forall j\ \forall v \quad Given(i, j, v) \Rightarrow Val(i, j, v)$$

**Codebase reference.** Implemented in `generate_given_clues(N, puzzle)`. For each pre-filled cell $(r, c)$ with value $v$, it generates a unit clause $[\mathrm{Val}(r, c, v)]$. The number of such clauses equals the number of given clues in the puzzle.

## 1.4 CNF Conversion: Step-by-Step Derivations

This section demonstrates the conversion of three non-trivial axioms from FOL to Conjunctive Normal Form (CNF), following the standard procedure:

1. Eliminate implications ($\Rightarrow$)
2. Move negations inward (De Morgan's laws)
3. Standardize variables
4. Skolemize (eliminate existential quantifiers)
5. Drop universal quantifiers
6. Distribute $\vee$ over $\wedge$ to obtain clausal form

To put it simple, dervivation means removing all signs, implications, etc... $\Rightarrow$ Removing the obvious facts (grounding)

### 1.4.1 Derivation 1: Axiom A1 (At-Least-One)

**A1:** $\forall i\ \forall j\ \exists v\ \mathbf{Val}(i, j, v)$

**Step 1: Eliminate implications.**
There are no implications ($\Rightarrow$).
**Step 2: Move negations inward.**
There are no negations.
**Step 3: Standardize variables.**
Variables $i, j, v$ are already distinct.
**Step 4: Skolemize.**
The variable $v$ is in the scope of $\forall i$ and $\forall j$, so we replace $v$ with a Skolem function $f(i, j)$:

$$\forall i\ \forall j\ \mathrm{Val}\Big(i, j, f(i, j)\Big)$$

However, the domain $\{1, \ldots, N\}$ is finite, we can expand the existential into a disjunction (the value for every cells):

$$\forall i\ \forall j\ \Big(\mathrm{Val}(i, j, 1) \vee \mathrm{Val}(i, j, 2) \vee \cdots \vee \mathrm{Val}(i, j, N)\Big)$$

**Step 5: Drop universal quantifiers.**

$$\mathrm{Val}(i, j, 1) \vee \mathrm{Val}(i, j, 2) \vee \cdots \vee \mathrm{Val}(i, j, N)$$

**Step 6: Clausal form.**
This is already a single clause. We obtain $N^2$ clauses:

$$\boxed{\Big\{\mathrm{Val}(i, j, 1),\ \mathrm{Val}(i, j, 2),\ \ldots,\ \mathrm{Val}(i, j, N)\Big\} \quad \text{for each } i, j}$$

**Example 1** (A1 for $N = 4$, cell $(1, 1)$)**.**

$$\{\mathit{Val}(1, 1, 1),\ \mathit{Val}(1, 1, 2),\ \mathit{Val}(1, 1, 3),\ \mathit{Val}(1, 1, 4)\}$$

*This single clause states: "Cell $(1, 1)$ must be assigned 1, 2, 3, or 4."*
*Total clauses generated: $4^2 = 16$ clauses.*

### 1.4.2 Derivation 2: Axiom A2 (At-Most-One)

**A2:** $\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad (\mathbf{Val}(i,j,v_1) \wedge \mathbf{Val}(i,j,v_2)) \Rightarrow v_1 = v_2$

**Step 1: Eliminate implications.**
Use $P \Rightarrow Q \equiv \neg P \vee Q$:

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \neg\Big(\mathrm{Val}(i,j,v_1) \wedge \mathrm{Val}(i,j,v_2)\Big) \vee (v_1 = v_2)$$

**Step 2: Move negations inward.**

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \neg\mathrm{Val}(i,j,v_1) \vee \neg\mathrm{Val}(i,j,v_2) \vee (v_1 = v_2)$$

**Step 3: Standardize variables.**
Variables already distinct.
**Step 4: Skolemize.**
No existential quantifiers.
**Step 5: Drop universal quantifiers.**

$$\neg\mathrm{Val}(i,j,v_1) \vee \neg\mathrm{Val}(i,j,v_2) \vee (v_1 = v_2)$$

**Step 6: Clausal form and grounding.**
We are grounding over a finite domain, we create for all $(i,j)$ and all pairs $(v_1, v_2)$ are $v_1 \neq v_2$. The equality $(v_1 = v_2)$ becomes False when $v_1 \neq v_2$ (only take one that $v_1 < v_2$ so the other side is removed):

$$\boxed{\Big\{\neg\mathrm{Val}(i,j,v_1),\ \neg\mathrm{Val}(i,j,v_2)\Big\} \quad \text{for each } i,j \text{ and each } v_1 < v_2}$$

**Example 2** (A2 for $N = 4$, cell $(1,1)$)**.** *The six binary clauses for cell* $(1,1)$*:*

$$\{\neg\, Val(1,1,1),\ \neg\, Val(1,1,2)\}$$
$$\{\neg\, Val(1,1,1),\ \neg\, Val(1,1,3)\}$$
$$\{\neg\, Val(1,1,1),\ \neg\, Val(1,1,4)\}$$
$$\{\neg\, Val(1,1,2),\ \neg\, Val(1,1,3)\}$$
$$\{\neg\, Val(1,1,2),\ \neg\, Val(1,1,4)\}$$
$$\{\neg\, Val(1,1,3),\ \neg\, Val(1,1,4)\}$$

*Each clause says: "Cell* $(1,1)$ *cannot simultaneously hold two different values."*
*Total clauses:* $16 \times \binom{4}{2} = 16 \times 6 = 96$ *clauses.*

### 1.4.3 Derivation 3: Axiom A3 (Row Uniqueness)

**A3:** $\forall i\, \forall j_1\, \forall j_2\, \forall v \quad (\mathbf{Val}(i, j_1, v) \wedge \mathbf{Val}(i, j_2, v) \wedge j_1 \neq j_2) \Rightarrow \bot$

**Step 1: Eliminate implications.**
$P \Rightarrow \bot \equiv \neg P$:

$$\forall i\, \forall j_1\, \forall j_2\, \forall v \quad \neg\Big(\mathrm{Val}(i, j_1, v) \wedge \mathrm{Val}(i, j_2, v) \wedge j_1 \neq j_2\Big)$$

**Step 2: Move negations inward.**

$$\forall i\, \forall j_1\, \forall j_2\, \forall v \quad \neg\mathrm{Val}(i, j_1, v) \vee \neg\mathrm{Val}(i, j_2, v) \vee (j_1 = j_2)$$

**Steps 3–4: Standardize and Skolemize.**
Variables already distinct.
**Step 5: Drop universal quantifiers.**

$$\neg\mathrm{Val}(i, j_1, v) \vee \neg\mathrm{Val}(i, j_2, v) \vee (j_1 = j_2)$$

**Step 6: Clausal form.**
After grounding for all $i, v$ and all pairs $(j_1, j_2)$ with $j_1 \neq j_2$, the equality disjunct becomes False (same logic as A2 but instead the same value, $(j_1 = j_2)$ will be remove):

$$\boxed{\Big\{\neg\mathrm{Val}(i, j_1, v),\ \neg\mathrm{Val}(i, j_2, v)\Big\} \quad \text{for each row } i,\ \text{value } v,\ \text{and } j_1 < j_2}$$

**Example 3** (A3 for $N = 4$, row 1, value 3)**.** *The six binary clauses:*

$$\{\neg\, Val(1, 1, 3),\ \neg\, Val(1, 2, 3)\}$$
$$\{\neg\, Val(1, 1, 3),\ \neg\, Val(1, 3, 3)\}$$
$$\{\neg\, Val(1, 1, 3),\ \neg\, Val(1, 4, 3)\}$$
$$\{\neg\, Val(1, 2, 3),\ \neg\, Val(1, 3, 3)\}$$
$$\{\neg\, Val(1, 2, 3),\ \neg\, Val(1, 4, 3)\}$$
$$\{\neg\, Val(1, 3, 3),\ \neg\, Val(1, 4, 3)\}$$

*Each says: "Value 3 cannot appear in two different columns of row 1."*
*Total clauses:* $4 \times 4 \times \binom{4}{2} = 96$ *clauses.*

### 1.4.4 Derivation 4: Axiom A4 (Horizontal Less-Than)

**A4:** $(\mathbf{LessH}(i,j) \wedge \mathbf{Val}(i,j,v_1) \wedge \mathbf{Val}(i,j{+}1,v_2)) \Rightarrow \mathbf{Less}(v_1,v_2)$

**Step 1: Eliminate implications.**

$$\neg\Big(\mathrm{LessH}(i,j) \wedge \mathrm{Val}(i,j,v_1) \wedge \mathrm{Val}(i,j{+}1,v_2)\Big) \vee \mathrm{Less}(v_1,v_2)$$

**Step 2: Move negations inward.**

$$\neg\mathrm{LessH}(i,j) \vee \neg\mathrm{Val}(i,j,v_1) \vee \neg\mathrm{Val}(i,j{+}1,v_2) \vee \mathrm{Less}(v_1,v_2)$$

**Steps 3–5: Standardize, Skolemize, drop $\forall$.**
No changes needed
**Step 6: Clausal form.**
After grounding, for each position $(i,j)$ where $\mathrm{LessH}(i,j)$ are at (known fact), $\neg\mathrm{LessH}(i,j)$ evaluates to False and is dropped. For each pair $(v_1,v_2)$:

- If $v_1 < v_2$: $\mathrm{Less}(v_1,v_2)$ is $\top$ (true by arithmetic), so the clause is trivially satisfied and can be dropped.
- If $v_1 \geq v_2$: $\mathrm{Less}(v_1,v_2)$ is $\bot$, so it is dropped, making:

$$\boxed{\Big\{\neg\mathrm{Val}(i,j,v_1),\ \neg\mathrm{Val}(i,j{+}1,v_2)\Big\} \quad \text{for each LessH position and } v_1 \geq v_2}$$

**Example 4** (A4 for $N=4$, constraint at $(1,1)$: cell $(1,1) <$ cell $(1,2)$)**.** *We enumerate all $(v_1,v_2)$ pairs with $v_1 \geq v_2$:*

$$\begin{aligned}
v_1 = 1: &\quad \{\neg\mathit{Val}(1,1,1),\ \neg\mathit{Val}(1,2,1)\}\\
v_1 = 2: &\quad \{\neg\mathit{Val}(1,1,2),\ \neg\mathit{Val}(1,2,1)\},\ \{\neg\mathit{Val}(1,1,2),\ \neg\mathit{Val}(1,2,2)\}\\
v_1 = 3: &\quad \{\neg\mathit{Val}(1,1,3),\ \neg\mathit{Val}(1,2,1)\},\ \{\neg\mathit{Val}(1,1,3),\ \neg\mathit{Val}(1,2,2)\},\ \{\neg\mathit{Val}(1,1,3),\ \neg\mathit{Val}(1,2,3)\}\\
v_1 = 4: &\quad \textit{(4 clauses for } v_2 \in \{1,2,3,4\}\textit{)}
\end{aligned}$$

*Total for this constraint:* $\sum_{v_1=1}^{4} v_1 = 1+2+3+4 = 10$ *clauses.*

## 1.5 Grounding the Knowledge Base

**Grounding** is the process of creating all universally variables in the finite domain $\{1,\ldots,N\}$ (or $\{0,\ldots,N{-}1\}$ in code), converting the FOL axioms into a large set of variable-free clauses.

### 1.5.1 Variable Encoding

Each ground atom $\mathrm{Val}(r,c,v)$ is mapped to a unique positive integer using the encoding:

$$\mathrm{ID}(r,c,v) = r \cdot N^2 + c \cdot N + (v-1) + 1$$

where $r, c \in \{0, \ldots, N-1\}$ and $v \in \{1, \ldots, N\}$.

The total number of propositional variables is $N^3$. For example, with $N = 4$, there are 64 variables; with $N = 9$, there are 729 variables.

### 1.5.2 Clause Count Analysis

The following table summarizes the number of clauses generated by each axiom for a puzzle of size $N$:

| **Axiom** | **Formula** | $N = 4$ | $N = 9$ |
|---|---|---|---|
| A1 (at-least-one) | $N^2$ | 16 | 81 |
| A2 (at-most-one) | $N^2 \cdot \binom{N}{2}$ | 96 | 2 916 |
| A3 (row unique) | $N \cdot N \cdot \binom{N}{2}$ | 96 | 2 916 |
| A6 (col unique) | $N \cdot N \cdot \binom{N}{2}$ | 96 | 2 916 |
| A4 (horiz $<$) | per constraint | varies | varies |
| A7 (horiz $>$) | per constraint | varies | varies |
| A8 (vert $<$) | per constraint | varies | varies |
| A9 (vert $>$) | per constraint | varies | varies |
| A5 (given clues) | # given cells | varies | varies |

Table 3: Number of CNF clauses per axiom.

For each inequality constraint, the number of forbidden pairs is $\frac{N(N+1)}{2}$ (for $\leq$ or $\geq$) or $\frac{N(N-1)}{2} + N$ clauses.

## 1.6 Implementation

The function `generate_ground_kb(puzzle)` in `cnf_generator.py` controls the entire grounding process:

1. Calls each axiom generator function (A1 through A9).
2. Collects all clauses into a single list.
3. Returns the clauses, total variable count ($N^3$), and per-axiom clause statistics.

The generated CNF can be:

- Solved directly using the `pysat` library's Glucose4 solver (`solve_with_pysat`).
- Exported to DIMACS format (`export_dimacs`) for verification with external SAT solvers.

# 2 Backward Chaining

## 2.1 From the FOL specification to the executable model

The project specification in `yeucau.txt` asks for a Prolog-style backward-chaining solver that can reason about predicates such as $Val(i, j, v)$, $Given(i, j, v)$, $LessH(i, j)$, $GreaterH(i, j)$, $LessV(i, j)$, $GreaterV(i, j)$, and $Less(v_1, v_2)$. At the specification level, the Futoshiki rules can be described in FOL-style terms, although the implementation does not explicitly encode quantifiers or a complete logical axiom system:

- every cell must receive exactly one value,
- every row and every column must contain pairwise distinct values,
- every horizontal and vertical inequality must be respected,
- every given clue must remain fixed.

A full FOL formalization would require quantified uniqueness and consistency constraints; the present solver does not materialize those formulas directly.

However, the executable solver in `Source/backward_chain.py` is *not* a purely declarative encoding of all these axioms as Horn clauses. Instead, it is a **hybrid design**:

1. a small Prolog-like SLD-resolution engine controls the proof search,
2. while the actual Futoshiki constraints are enforced procedurally by `compute_domain(...)` in `Source/futoshiki.py`.

This distinction is important for a technically accurate report. The implementation should therefore not be interpreted as a pure logical backward-chaining system. Its logical shell is goal-driven and Horn-clause based, but the row, column, and inequality semantics are delegated to Python built-ins rather than being expanded into a large explicit rule base.

| Concept in the specification | Executable representation | Where it is enforced |
|---|---|---|
| $Given(i, j, v)$ | fact `given(r,c,v)` | Loaded directly from the input grid into the backward-chaining knowledge base. |
| $Val(i, j, v)$ | query predicate `Val(R,C,V)` | Derived either from a `given` fact or from the built-ins `empty`, `domain`, and `safe_cell`. |
| Row / column uniqueness | not stored as explicit Horn predicates | Checked procedurally by `compute_domain(...)`. |
| Horizontal / vertical inequalities | not stored as explicit Horn predicates in the active BC solver | Also checked procedurally by `compute_domain(...)`. |
| Search state | Python dictionary `S` mapping `(row,col)` to value | Passed through the recursive predicate `solve_board(S,SF)`. |
| Complete solution | fully assigned dictionary state | Verified by `complete(S)` and then by `puzzle.is_valid(solution)`. |

Table 4: How the report-level FOL concepts are realized in the actual backward-chaining implementation.

A second implementation detail that must be stated explicitly is indexing. The report notation in `yeucau.txt` is naturally written with rows and columns numbered from 1 to $N$, but the executable code uses **0-based coordinates**. Therefore, the REPL query syntax shown by the program is `Val(0,1,?X)`, not `Val(1,2,?X)`.

## 2.2 Knowledge base and Horn clauses actually loaded by the solver

The active backward-chaining implementation begins in the later half of `Source/backward_chain.py`, where the file defines a small Prolog interpreter and a specialized knowledge base class `FutoshikiKB`. The knowledge base contains two kinds of facts:

### Static facts

- `domain(v)` for every $v \in \{1, 2, \ldots, N\}$,
- `given(r,c,v)` for every pre-filled cell in the puzzle.

### Rules for interactive cell queries

For individual backward-chaining queries, the solver loads the following Horn clauses:

$$Val(R, C, V) \leftarrow given(R, C, V)$$

$$Val(R, C, V) \leftarrow empty(R, C) \wedge domain(V) \wedge safe_cell(R, C, V)$$

The first rule is immediate: if a clue already fixes the value of a cell, the query succeeds directly from the fact base. The second rule applies only to empty cells. It first checks that the target cell is not pre-filled, then enumerates candidate values from the finite domain, and finally filters those values through `safe_cell`.

A subtle but important detail is that `safe_cell` checks legality against the *initial givens only*. In other words, the REPL query mode answers the question "which values are currently legal for this cell under the original puzzle constraints?", not necessarily "which value does this cell have in the unique final solution?" unless the puzzle already forces uniqueness at that location. Therefore, answers returned by `Val(R,C,V)` are local consistency checks only; they are not guaranteed to extend to a complete global solution.

### Rules for solving the whole board

For complete puzzle solving, the engine loads a second pair of Horn clauses:

$$solve_board(S, S) \leftarrow complete(S)$$

$$solve_board(S, S_F) \leftarrow select_unassigned(S, R, C) \wedge domain_val(S, R, C, V) \wedge assign(S, R, C, V, S_1) \wedge solve_$$

These are the rules that matter most for performance. The base case says that if the current state already assigns all $N^2$ cells, then the proof succeeds. The recursive rule says:

1. choose an unassigned cell,
2. enumerate legal values for that cell,
3. create a successor state,
4. recursively prove that the successor state can be extended to a full solution.

Unlike a purely symbolic theorem prover, the state $S$ here is not a set of logical atoms but a concrete Python dictionary. This is a pragmatic choice: it keeps the solver compact and makes constraint checking fast enough for realistic puzzles.

## 2.3 Core SLD-resolution engine: unification, substitution, and search order

The Prolog-style engine is centered around the datatypes `Var` and `Rule`, together with the helper constructor `lit(name,*args)`. A literal is represented as a tuple of the form

$$(predicate_name,\ argument_tuple).$$

The inference process relies on four standard operations.

### Variable walking and substitution

The helper `_walk(term, subst)` follows a chain of variable bindings until it reaches a non-variable term. Then `_substitute(...)` recursively applies that substitution to tuples. This is the mechanism by which partially instantiated goals are rewritten as the proof proceeds.

### Occurs check and unification

The solver implements an explicit occurs check in `_occurs_check(var,term,subst)`. During unification, if a variable would end up containing a term that already contains that same variable, the match fails. This avoids cyclic substitutions and keeps the term system well formed.

One implementation-specific optimization deserves mention: if both terms are Python dictionaries, the unifier compares them directly by value. This matters because whole-board solving passes states such as `S` and `S1` through the proof process.

### Standardize-apart

Before a rule is used, `_standardize_apart(rule)` renames its variables to fresh names. This is the standard Prolog precaution that prevents variable capture when the same rule is reused multiple times along different proof branches.

### Depth-first, left-to-right goal expansion

The engine routine `_solve_and(goals, subst)` follows the classic SLD discipline:

1. take the leftmost goal,
2. apply the current substitution,
3. if the predicate is built-in, execute the corresponding Python procedure,
4. otherwise scan the rule base for heads with matching predicate name and arity,
5. unify the rule head with the goal,
6. prepend the rule body to the remaining goals,
7. continue recursively in depth-first order.

Therefore, the solver is genuinely goal-driven, but it does **not** implement memoization, tabling, or duplicate-state caching. If two proof branches lead to logically similar subproblems, the engine may recompute them.

**Operational View of the Implemented SLD Search**

Let the current goal list be $[G_1, G_2, \ldots, G_k]$ and let $\theta$ be the current substitution.

1. Replace $G_1$ by $G_1\theta$.
2. If $G_1$ is a built-in predicate, execute its Python handler and recurse on the remaining goals for each produced substitution.
3. Otherwise, for every rule $H \leftarrow B_1 \wedge \cdots \wedge B_m$ whose standardized head unifies with $G_1\theta$, recurse on
$$[B_1, \ldots, B_m, G_2, \ldots, G_k]$$
under the extended substitution.

Search succeeds when the goal list becomes empty.

## 2.4 Where the actual Futoshiki reasoning happens: MRV, domain filtering, and forward checking

The logical control layer alone would be far too weak if every constraint had to be rediscovered by naive backtracking. The reason the implementation is still usable in practice is that several CSP-style heuristics are embedded in the built-ins.

### The role of `compute_domain(...)`

The function `compute_domain(puzzle, assignment, r, c)` in `Source/futoshiki.py` is the semantic heart of the solver. Given a partial assignment, it removes values that violate:

- row uniqueness,
- column uniqueness,
- horizontal inequalities,
- vertical inequalities.

Therefore, the backward-chaining engine does not prove these constraints by symbolic derivation. Instead, it asks `compute_domain(...)` which values remain legal at a cell under the current partial assignment.

### MRV variable ordering

The built-in `select_unassigned(S,R,C)` scans all still-empty cells and chooses the one with the smallest remaining domain. This is the classical **Minimum Remaining Values (MRV)** heuristic. If a cell already has an empty domain, the built-in fails immediately and the whole proof branch is cut.

This built-in is crucial because it turns the proof tree from a blind left-to-right filling order into a fail-first search strategy. In difficult puzzles, choosing the most constrained cell first can reduce the branching factor dramatically.

#### Ordered value generation

Once MRV has selected a cell, `domain_val(S,R,C,V)` enumerates the legal values returned by `compute_domain(...)` in sorted order. This does not change correctness, but it makes the solver deterministic and easier to analyze experimentally.

#### Forward checking inside `assign`

The built-in `assign(S,R,C,V,S1)` performs three operations:

1. copy the current dictionary state,
2. assign value $V$ to cell $(R, C)$,
3. check whether any still-unassigned neighbor in the same row or column now has an empty domain.

If such a domain wipeout occurs, the branch fails *before* deeper recursion. This is forward checking. The same built-in also increments `nodes_explored`, which is the main search statistic returned by the backward-chaining solver.

#### Final validation

Even after a proof succeeds, `solve_backward_chaining(...)` still validates the produced dictionary by calling `puzzle.is_valid(solution)`. This final guard is good engineering practice: it ensures that a returned state is not merely complete, but also globally consistent with the puzzle constraints.

The consequence is clear: the implemented solver is best described as **SLD-resolution for search control + CSP propagation for constraint enforcement**. This hybrid view is more accurate than calling it a purely symbolic FOL prover or a purely logical backward-chaining engine.

From a complexity perspective, the procedure remains exponential in the worst case. If the board has size $N \times N$, the search depth is at most $N^2$, and a loose upper bound on the branching factor is $N$, so the naive worst-case search cost is on the order of $O(N^{N^2})$. MRV, domain filtering, and forward checking reduce this dramatically in many practical cases, but they do not change the worst-case exponential nature of the search.

## 2.5 Code-faithful pseudocode of the implemented solver

The following pseudocode is intentionally aligned with the actual implementation rather than with an abstract textbook presentation.

### ASK

```
function ASK(query):
    yield from SOLVE_AND([query], empty_substitution)
```

### SOLVE_AND

```
function SOLVE_AND(goals, theta):
    if goals is empty:
        yield theta
        return

    (pred, args) = goals[0]
    args = SUBSTITUTE(args, theta)
    rest = goals[1:]

    if pred is a built-in predicate:
        for each theta1 produced by BUILTIN(pred, args, theta):
            yield from SOLVE_AND(rest, theta1)
        return

    for each rule in KB:
        if predicate name or arity does not match:
            continue
        rule' = STANDARDIZE_APART(rule)
        theta1 = UNIFY(head(rule').args, args, theta)
        if theta1 succeeds:
            new_goals = body(rule') + rest
            yield from SOLVE_AND(new_goals, theta1)
```

### SOLVE_BACKWARD_CHAINING

```
function SOLVE_BACKWARD_CHAINING(puzzle):
    kb = FutoshikiKB(puzzle)
    S0 = build_initial_assignment(puzzle)
    Final = fresh variable

    query = solve_board(S0, Final)

    for each answer theta in ASK(query):
        SF = WALK(Final, theta)
        if SF is a dictionary and puzzle.is_valid(SF):
            return SF

    return failure
```

**SELECT_UNASSIGNED**

```
built-in SELECT_UNASSIGNED(S):
    choose empty cell with minimum |compute_domain(puzzle, S, r, c)|
```

**DOMAIN_VAL**

```
built-in DOMAIN_VAL(S, R, C):
    return sorted(compute_domain(puzzle, S, R, C))
```

**ASSIGN**

```
built-in ASSIGN(S, R, C, V):
    S1 = copy(S)
    S1[(R, C)] = V
    if any unassigned cell in row R or column C has empty domain:
        fail
    else:
        nodes_explored += 1
        return S1
```

The pseudocode is separated by function so each block is shorter and easier to place across pages.

This pseudocode highlights the most important implementation fact: the recursion is expressed declaratively as `solve_board`, but the expensive logical content of the puzzle is discharged procedurally inside `compute_domain` and the built-ins that call it.

## 2.6 Step-by-step simulation on a $4 \times 4$ puzzle

To illustrate the actual control flow of the implemented backward-chaining solver, we use two representative $4 \times 4$ benchmark instances from the project input format and then trace `solve_backward_chaining(...)` on `input-01.txt`. Unlike the forward-chaining simulation, this trace is not organized around repeated global propagation passes. Instead, it follows the recursive goal

$$solve_board(S_0, S_F)$$

and shows how the built-ins `select_unassigned`, `domain_val`, and `assign` drive the search.

### Visualized $4 \times 4$ inputs from the benchmark format

The following boards are drawn directly from the provided grid data and constraint matrices.

**input-01.txt**

| | | | |
|---|---|---|---|
| 1 | ? | 3 | ? |
| 3 | 4 | ? | 1 |
| 2 | ? | 4 | 3 |
| ? | 3 | ? | 2 |

**input-03.txt**

$$
\begin{array}{ccccccc}
2 & & 4 & > & ? & > & 1 \\
\vee & & & & \wedge & & \wedge \\
1 & & ? & < & 4 & > & 3 \\
\wedge & & \wedge & & \vee & & \vee \\
4 & > & 3 & > & 1 & < & ? \\
\vee & & & & \wedge & & \wedge \\
? & > & 1 & < & 2 & < & 4
\end{array}
$$

### Detailed trace on `input-01.txt`

The initial puzzle is:

**Initial state for the BC trace**

| | | | |
|---|---|---|---|
| 1 | ? | 3 | ? |
| 3 | 4 | ? | 1 |
| 2 | ? | 4 | 3 |
| ? | 3 | ? | 2 |

The initial assignment $S_0$ built from the givens contains 10 fixed cells, so there are 6 empty cells:

$$(0,1),\ (0,3),\ (1,2),\ (2,1),\ (3,0),\ (3,2).$$

Before any recursive assignment is made, the solver evaluates the legal domains of all empty cells through `compute_domain(...)`:

$$
\begin{aligned}
D(0,1) &= \{2\}, \quad D(0,3) = \{4\}, \quad D(1,2) = \{2\}, \\
D(2,1) &= \{1\}, \quad D(3,0) = \{4\}, \quad D(3,2) = \{1\}.
\end{aligned}
$$

Every remaining variable is already a singleton. Therefore, the MRV heuristic cannot distinguish them by domain size and simply returns the first unassigned cell encountered in row-major order. This detail is important because it explains the exact sequence of assignments observed in the real execution trace.

**Step 1: select** $(0, 1)$ **and assign value** $2$

The first recursive call applies

$$select_unassigned(S_0, R, C)$$

which returns $(R, C) = (0, 1)$. Then

$$domain_val(S_0, 0, 1, V)$$

enumerates the single legal value $V = 2$. The built-in

$$assign(S_0, 0, 1, 2, S_1)$$

creates a new dictionary state and performs forward checking on row 0 and column 1. No domain wipeout occurs, so the branch survives.

After this step, the remaining empty-cell domains are:

$$D(0, 3) = \{4\}, \quad D(1, 2) = \{2\}, \quad D(2, 1) = \{1\}, \quad D(3, 0) = \{4\}, \quad D(3, 2) = \{1\}.$$

The callback trace records:

```
[BC] Set (0,1) = 2 | Explored:  1
```

**Step 2: select** $(0, 3)$ **and assign value** $4$

The next call to `select_unassigned` scans row-major order again and now finds $(0, 3)$ as the first still-empty cell. Its domain is again a singleton:

$$D(0, 3) = \{4\}.$$

Thus the solver commits to

$$assign(S_1, 0, 3, 4, S_2).$$

Forward checking succeeds, and the search continues with four empty cells left.

The execution trace records:

```
[BC] Set (0,3) = 4 | Explored:  2
```

**Step 3: select** $(1, 2)$ **and assign value** $2$

The next unassigned cell in scan order is $(1, 2)$, and its legal domain is

$$D(1, 2) = \{2\}.$$

Therefore the only branch generated by `domain_val` is $V = 2$, producing the successor state

$$assign(S_2, 1, 2, 2, S_3).$$

Again, forward checking succeeds immediately because no row or column neighbor loses its last remaining value.

The trace message is:

```
[BC] Set (1,2) = 2 | Explored:  3
```

**Step 4: select** $(2, 1)$ **and assign value** $1$

At this point the unfinished cells are $(2, 1)$, $(3, 0)$, and $(3, 2)$. The solver chooses $(2, 1)$ first because it is the first one reached in row-major order. Its domain is

$$D(2, 1) = \{1\}.$$

So the recursive proof extends to

$$assign(S_3, 2, 1, 1, S_4).$$

The branch remains consistent, and only the last row still contains empty cells.

The trace message is:

```
[BC] Set (2,1) = 1 | Explored:  4
```

**Step 5: select** $(3, 0)$ **and assign value** $4$

The next chosen cell is $(3, 0)$ with domain

$$D(3, 0) = \{4\}.$$

The solver therefore applies

$$assign(S_4, 3, 0, 4, S_5).$$

After that assignment, only one empty cell remains in the puzzle.

The trace message is:

```
[BC] Set (3,0) = 4 | Explored:  5
```

**Step 6: select $(3, 2)$ and assign value $1$**

The final unassigned cell is $(3, 2)$, whose domain is

$$D(3, 2) = \{1\}.$$

The last recursive extension is therefore

$$assign(S_5, 3, 2, 1, S_6).$$

Now the state contains all 16 assignments, so the base rule

$$solve_board(S, S) \leftarrow complete(S)$$

succeeds, and the proof unwinds successfully.

The final callback message is:

```
[BC] Set (3,2) = 1 | Explored:  6
```

### Final solution and interpretation

The completed board is:

**Final solved board for `input-01.txt`**

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 3 | 4 | 2 | 1 |
| 2 | 1 | 4 | 3 |
| 4 | 3 | 1 | 2 |

This run is a particularly clean example of the implemented backward-chaining strategy:

- MRV always sees singleton domains,
- `domain_val` therefore generates exactly one candidate at each level,
- `assign` never triggers a forward-checking failure,
- the solver reaches a complete valid state after exactly 6 explored nodes.

In other words, this instance behaves almost like deterministic logical deduction, even though the solver is fundamentally a depth-first search procedure. The benchmark data confirms this interpretation: for `input-01.txt`, backward chaining solves the puzzle in approximately 0.0006 seconds with 6 explored nodes.

This example is intentionally simple, so it does not illustrate backtracking behavior. Its value is mainly explanatory: it makes the control flow of `solve_board` easy to follow.

### A second $4 \times 4$ trace: `input-03.txt`

A second small trace can be extracted from `input-03.txt`, whose initial board is

**input-03.txt**

$$
\begin{array}{ccccccc}
2 & & 4 & > & ? & > & 1 \\
\vee & & & & \wedge & & \wedge \\
1 & & ? & < & 4 & > & 3 \\
\wedge & & \wedge & & \vee & & \vee \\
4 & > & 3 & > & 1 & < & ? \\
\vee & & & & \wedge & & \wedge \\
? & > & 1 & < & 2 & < & 4
\end{array}
$$

with empty cells

$$(0,2),\ (1,1),\ (2,3),\ (3,0).$$

The initial domains are already all singletons:

$$D(0,2) = \{3\}, \quad D(1,1) = \{2\}, \quad D(2,3) = \{2\}, \quad D(3,0) = \{3\}.$$

Therefore, the solver again follows row-major MRV tie-breaking and never branches.

**Step 1.** The solver selects $(0,2)$ and assigns 3:

$$assign(S_0, 0, 2, 3, S_1).$$

The trace records `[BC] Set (0,2) = 3 | Explored: 1`.

**Step 2.** Next it selects $(1,1)$ and assigns 2:

$$assign(S_1, 1, 1, 2, S_2).$$

The trace records `[BC] Set (1,1) = 2 | Explored: 2`.

**Step 3.** Then it selects $(2,3)$ and assigns 2:

$$assign(S_2, 2, 3, 2, S_3).$$

The trace records `[BC] Set (2,3) = 2 | Explored: 3`.

**Step 4.** Finally it selects $(3, 0)$ and assigns 3:

$$assign(S_3, 3, 0, 3, S_4).$$

The trace records `[BC] Set (3,0) = 3 | Explored: 4`.

The completed solution is

**Final solved board for `input-03.txt`**

| | | | | | | |
|---|---|---|---|---|---|---|
| 2 | | 4 | > | 3 | > | 1 |
| ∨ | | | | ∧ | | ∧ |
| 1 | | 2 | < | 4 | > | 3 |
| ∧ | | ∧ | | ∨ | | ∨ |
| 4 | > | 3 | > | 1 | < | 2 |
| ∨ | | | | ∧ | | ∧ |
| 3 | > | 1 | < | 2 | < | 4 |

and the benchmark file reports approximately 0.0005 seconds and 4 explored nodes for backward chaining on `input-03.txt`. Like `input-01.txt`, this instance is too constrained to expose the harder branching behavior of the algorithm, but it provides a second code-faithful sanity check for the execution trace.

## 2.7 Tested backward-chaining outputs on representative instances

To ground the discussion in actual executions, the backward-chaining solver was run on a small instance and on a larger inequality-rich instance. The resulting boards were saved as text outputs and rendered into figures for the report.

### Small instance: `input-01.txt`

Figure 1 shows the solved $4 \times 4$ puzzle produced by backward chaining. This case is useful because the search tree is shallow and the effect of MRV is easy to see: the benchmark file reports only 6 explored nodes and a runtime of approximately 0.0006 seconds.

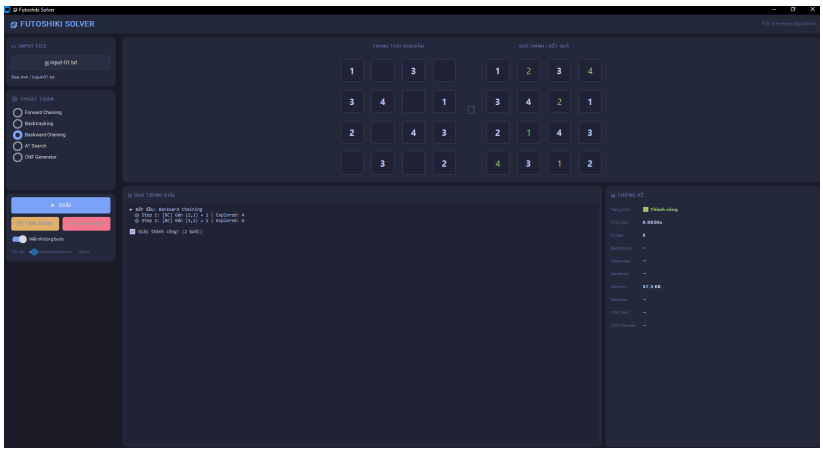



Figure 1: Backward-chaining solution on `input-01.txt`.

For this small puzzle, backward chaining behaves almost ideally: the constraints are tight, legal domains collapse quickly, and forward checking removes bad branches before they expand.

### Larger instance: `input-10.txt`

Figure 2 shows a solved $9 \times 9$ board with many inequality constraints. This is a better demonstration of the actual implementation because it exercises the recursive `solve_board` rule for a much deeper search tree. According to `Source/Results/benchmark.csv`, this instance required 456 explored nodes and about 0.0525 seconds.

Figure 2: Backward-chaining solution on `input-10.txt`.

The board in Figure 2 also illustrates why preserving inequality symbols in the output is useful. The final assignment alone is not enough to communicate correctness visually; the embedded $<$, $>$, $\wedge$, and $v$ relations show that the solution satisfies the original puzzle structure.

## 2.8 Experimental evidence from the benchmark data

The backward-chaining implementation was also evaluated in the project benchmark file `Source/Results/benchmark.csv`. Table 5 summarizes several representative cases.

| Input | Status | Time (s) | Nodes explored | Peak memory (KB) |
|---|---|---|---|---|
| `input-01.txt` | solved | 0.000586 | 6 | 68.71 |
| `input-03.txt` | solved | 0.000484 | 4 | 52.91 |
| `input-10.txt` | solved | 0.052535 | 456 | 2414.55 |
| `input-12.txt` | solved | 9.318154 | 89565 | 3538.17 |
| `input-11.txt` | timeout | 600.0 | – | – |

Table 5: Selected benchmark results for backward chaining extracted from `Source/Results/benchmark.csv`.

Several conclusions follow immediately.

- On easy and medium-sized puzzles, backward chaining is practical and often quite fast.
- On harder instances, the search tree can still explode despite MRV and forward checking.
- The implementation is clearly *not* polynomial-time in practice; it remains an exponential search procedure with good pruning.

A particularly instructive contrast is between `input-10.txt` and `input-12.txt`. Both are $9 \times 9$ puzzles, but `input-10.txt` has 21 givens and 72 inequality relations, whereas `input-12.txt` has only 11 givens and 43 inequality relations. The much poorer performance on `input-12.txt` suggests that weaker early constraint propagation leaves larger domains ambiguous for longer, so the DFS-style search has to explore far more branches before reaching a solution.

This also helps explain why forward chaining can outperform backward chaining on some structured instances. For example, the benchmark file reports only 61 FC nodes on `input-10.txt`, compared with 456 BC nodes, because forward propagation can exploit globally forced consequences earlier instead of rediscovering them along multiple depth-first branches.

Another point that should be stated honestly in the report is instrumentation. The backward-chaining solver currently returns only `time` and `nodes`. In the benchmark CSV, the `backtracks` field is recorded as `-1` and the `inferences` field as `0` for BC rows, because those quantities are not explicitly counted by the present implementation. Therefore, comparisons involving "number of inferences" must be phrased carefully.

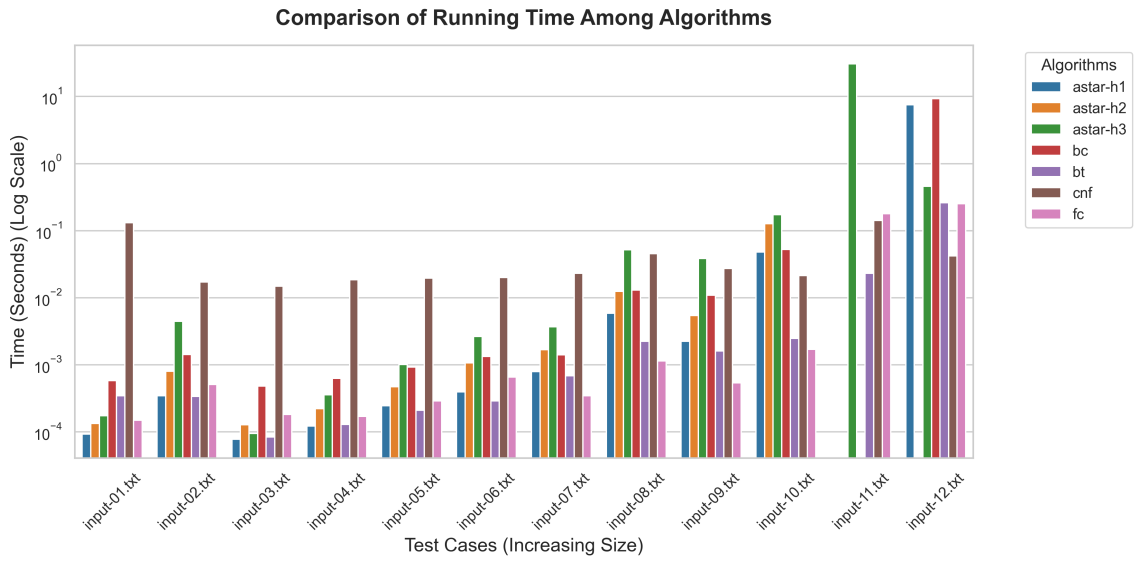



Figure 3: Running-time comparison across algorithms. The backward-chaining curve is part of the generated benchmark charts and shows strong variance between easy and difficult inputs.

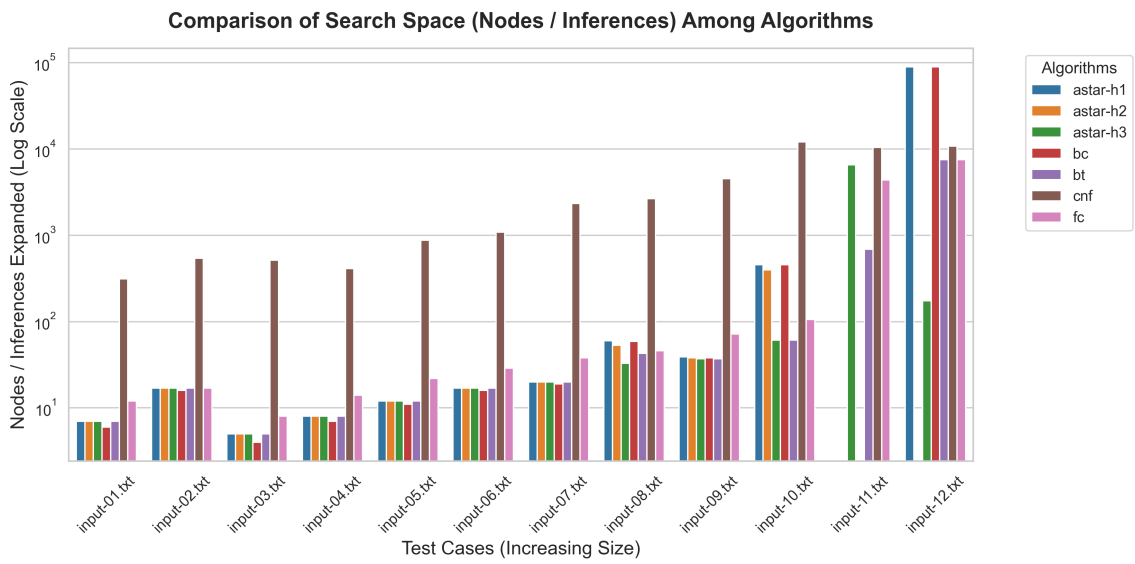



Figure 4: Comparison of search effort across algorithms. For backward chaining, the plotted workload corresponds to explored nodes recorded by the solver.

The charts reinforce the same interpretation as Table 5. Backward chaining is attractive when local constraints are informative enough to make MRV and forward checking decisive, but it deteriorates sharply on puzzles whose remaining domains stay ambiguous for many levels of recursion.

## 2.9 Advantages and limitations of this specific backward-chaining design

### Advantages

- **Faithful to the project requirement at the control level:** the solver is genuinely Prolog-like in its use of Horn clauses, unification, substitution, and depth-first SLD-style recursion.
- **Supports interactive querying:** the program exposes a REPL for queries of the form `Val(Row, Col, ?X)`, which directly addresses the specification's requirement to query individual cell values.
- **Good pruning in practice:** MRV and forward checking prevent the proof tree from degenerating into completely blind backtracking on many instances.
- **Simple integration with the rest of the codebase:** the backward-chaining engine reuses `build_initial_assignment`, `compute_domain`, and `puzzle.is_valid`, so all solvers share the same notion of legality.
- **Explainable control flow:** every successful solution corresponds to a chain of subgoals created by the recursive predicate `solve_board`.

**Limitations**

- **Not a pure FOL realization of all puzzle axioms:** row, column, and inequality constraints are not represented as explicit Horn clauses in the active solver, and quantified uniqueness constraints are not encoded directly.
- **No memoization or tabling:** repeated subproblems may be recomputed, which hurts hard instances.
- **Query mode is local, not global:** `Val(R,C,V)` checks legality under the initial givens only, so a returned value need not be extendable to a full solution.
- **Search can still explode:** the timeout on `input-11.txt` and the heavy search cost on `input-12.txt` show that the heuristic pruning is helpful but not sufficient to eliminate exponential behavior.
- **Limited statistics:** the implementation records explored nodes, but it does not expose precise counts for logical inferences or backtracks.

Overall, backward chaining is a strong fit for this project when described honestly: it is a **goal-driven Prolog-style search procedure wrapped around procedural constraint propagation**. That makes it much more practical than a naive symbolic proof procedure, while still preserving the flavor of logical inference required by the assignment.

# 3 Forward Chaining

## 3.1 Theoretical Foundation and Definitions

Forward chaining is a **data-driven** (bottom-up) inference method used in rule-based systems. Starting from known facts, the engine repeatedly applies inference rules to derive new facts until no more new information can be produced.

### 3.1.1 Key Concepts

- **Modus Ponens:** The fundamental inference rule. Given a fact $P$ and a rule $P \Rightarrow Q$, we derive the new fact $Q$. In forward chaining, every derivation step is an application of Modus Ponens.
- **Fixpoint:** The state where no inference rule can produce any new fact. Formally, let $T_P$ denote one full pass of applying all rules to the current fact set. The fixpoint is reached when $T_P(\text{facts}) = \text{facts}$ (no new facts are added). The forward chaining loop terminates exactly at this point.
- **Soundness:** Forward chaining is *sound*: every derived fact logically follows from the initial facts and the rules. It never produces incorrect conclusions.
- **Incompleteness:** Forward chaining is *incomplete* for general problem solving: when no rule fires (i.e., no cell value is logically forced), it cannot make progress—even if a solution exists. This is because FC never "guesses" or tries a value speculatively.

### 3.1.2 Data-Driven vs. Goal-Driven Inference

Forward chaining is the opposite of backward chaining. While backward chaining starts from a query ("what is the value of cell $(r, c)$?") and works backward to find supporting facts, forward chaining starts from all known facts and works forward, deriving everything it can. This makes it particularly effective when many initial facts (given clues) trigger cascading derivations.

## 3.2 Application to the Futoshiki Problem

For the Futoshiki puzzle, the forward chaining model maps as follows:

| FC Concept | Futoshiki Mapping |
|---|---|
| Facts | Cell assignments: $\text{Val}(r, c, v)$ means cell $(r, c)$ holds value $v$. Initially, these are the given clues (Axiom A5). |
| Rules | The grounded FOL axioms A1–A9 from Section 1. Each rule removes impossible values from cell domains or forces cell assignments. |
| Domains | For each unassigned cell $(r, c)$, the set $D(r, c) \subseteq \{1, \ldots, N\}$ of candidate values not yet ruled out. |
| Fixpoint | The state where no rule can shrink any domain further. Either all cells are assigned (solved) or FC is stuck (fallback to BT). |

Table 6: Forward chaining concepts mapped to the Futoshiki puzzle.

### 3.2.1 Knowledge Base Structure

Our implementation uses a `KnowledgeBase` class (defined in `forward_chain.py`) that stores two things:

- **facts**: a dictionary $\{(r, c) \mapsto v\}$ of cells whose values are already known.
- **domains**: a dictionary $\{(r, c) \mapsto \{v_1, v_2, \ldots\}\}$ of candidate values for each cell that has not been assigned yet.

At the start, we load the given clues into `facts` (this is Axiom A5: $\text{Given}(i, j, v) \Rightarrow \text{Val}(i, j, v)$). Every empty cell gets the full domain $\{1, \ldots, N\}$. Then we run a one-time propagation pass to remove obviously impossible values based on the initial clues.
**Codebase reference:** The `KnowledgeBase` class is defined in `forward_chain.py` (lines 12–120). Fact storage uses a plain dictionary, and domains use a dictionary of Python sets for efficient add/remove operations.

## 3.3 Inference Rules and Their FOL Basis

We apply five rules in each iteration. Each rule is rooted in the FOL axioms from Section 1, but implemented as direct domain operations for efficiency. The following table summarizes the mapping:

| Rule | Name | Axiom(s) | Action |
|---|---|---|---|
| 1 | Singleton Domain | A1 + A2 | $\lvert D(r,c)\rvert = 1 \Rightarrow$ assign $v$ |
| 2 | Row Uniqueness | A3 | Remove $v$ from same-row domains |
| 3 | Column Uniqueness | A6 | Remove $v$ from same-column domains |
| 4 | Hidden Single | A3 + A6 | If $v$ fits only one cell in a unit, assign it |
| 5 | Inequality Chain | A4, A7, A8, A9 | BFS-based bound propagation |

Table 7: Mapping of FC inference rules to FOL axioms from Section 1.

#### 3.3.1 Rule 1: Singleton Domain (from A1 + A2)

If a cell's domain has shrunk to exactly one value, that must be the cell's value:

$$|D(r,c)| = 1 \implies \text{Val}(r,c,v) \text{ where } \{v\} = D(r,c)$$

One important implementation detail: instead of assigning the first singleton we find and then restarting the scan (which wastes $\mathcal{O}(N^2)$ work each time), we collect **all** singletons in a single pass and process them together as a batch. After processing the batch, we scan again because previous assignments may have created new singletons through propagation.
**Codebase reference:** Implemented in `_apply_singleton_rule()` in `forward_chain.py`.

#### 3.3.2 Rule 2 and 3: Row and Column Uniqueness (A3, A6)

When we learn $\text{Val}(r,c,v)$, we remove $v$ from the domain of every other cell in the same row (A3) and same column (A6). These two rules are applied inside the propagation function every time a new fact is added.
**Codebase reference:** Implemented in `_propagate_single_fact()` in `forward_chain.py`.

#### 3.3.3 Rule 4: Hidden Single (A3 + A6 combined)

If we look at a row and find that some value $v$ can only go in one cell, then that cell must hold $v$. Same idea for columns. For example, if in row 2 only cell $(2,3)$ can still hold the value 4, then $\text{Val}(2,3,4)$.

Just like the singleton rule, we scan *all* rows and columns in one pass to find *all* hidden singles, then process them as a batch. This avoids the $\mathcal{O}(N^3)$ penalty of restarting after each individual finding.
**Codebase reference:** Implemented in `_apply_hidden_single_rule()` in `forward_chain.py`.

#### 3.3.4 Rule 5: Queue-Based Inequality Propagation (A4, A7, A8, A9)

This is the most interesting rule. When a cell's value or domain bounds change, we propagate inequality constraints to its neighbors. The key insight is that we use a **BFS queue** to handle chains of inequalities automatically.

For a constraint like $(r, c) < (r, c{+}1)$, the propagation logic uses bounds:

- If $(r, c)$ is assigned to value $v$, then every value in $D(r, c{+}1)$ must be $> v$.
- If $(r, c)$ is unassigned with domain $D$, then every value in $D(r, c{+}1)$ must be $> \min(D)$.
- Conversely, every value in $D(r, c)$ must be $< \max(D(r, c{+}1))$.

**Why the queue matters: handling $A < B < C$**

Consider a chain $A < B < C$. Suppose we just learned $A = 1$.

1. Cell $A$ enters the propagation queue.
2. We process $A$: filter $D(B)$ to keep only values $> 1$. If $D(B)$ actually changed (say from $\{1, 2, 3\}$ to $\{2, 3\}$), we add $B$ to the queue.
3. We process $B$: now $\min(D(B)) = 2$, so we filter $D(C)$ to keep only values $> 2$.

Without the queue, only the direct neighbor $B$ would be updated in one pass, and we would need to wait for the outer loop iteration to eventually propagate to $C$. The queue handles arbitrarily long chains in a single propagation call.

**Codebase reference:** The queue-based propagation is implemented in the second half of `_propagate_single_fact()` in `forward_chain.py`. The function `_propagate_bounds()` checks all four directions (left, right, up, down) for inequality constraints.

## 3.4 Algorithm Structure (Pseudocode)

#### 3.4.1 The Main Loop

The top-level forward-chaining procedure repeats the inference rules until a fixpoint:

**Pseudocode of the FC main loop (aligned with forward_chain.py)**

```
function FORWARD_CHAIN(puzzle):
    KB = new KnowledgeBase(puzzle)
    // KB automatically propagates given clues

    repeat:
        new_facts = 0
        new_facts += APPLY_SINGLETON_RULE(KB)
        new_facts += APPLY_HIDDEN_SINGLE_RULE(KB)
    until new_facts == 0   // fixpoint

    if all N*N cells are in KB.facts:
        return KB.facts     // FC solved it alone
    else:
        // FC got stuck, hand off to Backtracking
        return BT_FALLBACK(puzzle, KB.facts, KB.domains)
```

Each call to `APPLY_SINGLETON_RULE` internally loops: it collects all current singletons, adds them as facts (which triggers propagation of row/column uniqueness and inequality chains), checks for new singletons created by propagation, and repeats. The function returns only when no more singletons exist.

### 3.4.2 Propagation Detail

Every time a new fact $\text{Val}(r, c, v)$ is added, the following propagation sequence runs:

**Propagation when a fact is added (aligned with _propagate_single_fact)**

```
function PROPAGATE(KB, r, c, value):
    // A3: remove value from all cells in same row
    for each (r, c') where c' != c and (r,c') in domains:
        D(r, c').discard(value)

    // A6: remove value from all cells in same column
    for each (r', c) where r' != r and (r',c) in domains:
        D(r', c).discard(value)

    // A4/A7/A8/A9: inequality chain propagation
    queue = [(r, c)]
    visited = empty set
    while queue not empty:
        (cr, cc) = queue.popleft()
        if (cr, cc) in visited: skip
        visited.add((cr, cc))
        changed = PROPAGATE_BOUNDS(cr, cc)
        for cell in changed:
            queue.append(cell)
```

### 3.4.3 Complexity Estimate

Each propagation pass visits at most $\mathcal{O}(N)$ cells per row/column update and $\mathcal{O}(N^2)$ cells in the inequality BFS. The main loop runs at most $\mathcal{O}(N^2)$ iterations (one per cell assignment). Therefore, the total time complexity of the FC phase is approximately $\mathcal{O}(N^4)$ in the worst case. In practice, the fixpoint is reached much faster because most propagation passes affect only a small number of cells.

## 3.5 Backtracking Fallback

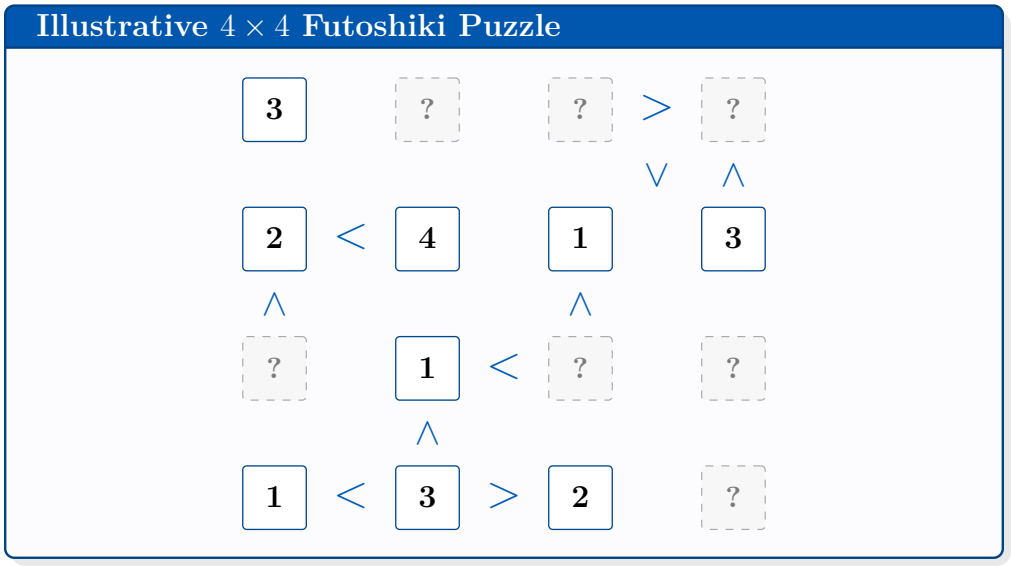
Forward chaining is **incomplete** by nature. On harder puzzles, it may reach a fixpoint where no rule fires but some cells remain empty. When this happens, our solver transitions to a Backtracking phase (the same BT solver described in the next section). The domains that FC already reduced are passed directly to BT, so BT searches a much smaller space than it would from scratch.

In the output, we mark the status as `ok(fc+bt)` when fallback was needed, and plain `ok` when FC solved the puzzle entirely on its own. The statistics also separate FC inferences from BT nodes so we can see exactly how much work each phase did.

## 3.6 Step-by-Step Simulation on a $4 \times 4$ Puzzle

We trace our FC implementation on `input-01.txt`, the same $4 \times 4$ puzzle used in the A* simulation (Section 5). The initial grid is:

**Illustrative $4 \times 4$ Futoshiki Puzzle**




There are 9 given cells and 7 empty cells: $(0,1)$, $(0,2)$, $(0,3)$, $(2,0)$, $(2,2)$, $(2,3)$, $(3,3)$.

#### Phase 1: Initial Propagation (`_propagate_all_facts`)

When the `KnowledgeBase` is created, it propagates all 9 given clues. We trace the key domain changes:

1. **Propagate** $(0,0) = 3$: Remove 3 from row 0 and column 0.

- $D(0,1) : \{1,2,3,4\} \to \{1,2,4\}, \quad D(0,2) \to \{1,2,4\}, \quad D(0,3) \to \{1,2,4\}$
- $D(2,0) : \{1,2,3,4\} \to \{1,2,4\}$

2. **Propagate** $(1,0) = 2$: Remove 2 from column 0. Then apply inequality $v_con[1][0] = 1$, which means $(1,0) < (2,0)$.

   - $D(2,0) : \{1,2,4\} \xrightarrow{\text{col}} \{1,4\} \xrightarrow{v>2} \{\mathbf{4}\}$

   The inequality queue adds $(2,0)$ since its domain changed, but $(2,0)$ has no further inequality neighbors, so the queue empties.

3. **Propagate** $(1,1) = 4$: Remove 4 from column 1.

   - $D(0,1) : \{1,2,4\} \to \{\mathbf{1},\mathbf{2}\}$

4. **Propagate** $(1,2) = 1$: Remove 1 from column 2.

   - $D(0,2) : \{1,2,4\} \to \{2,4\}, \quad D(2,2) : \{1,2,3,4\} \to \{2,3,4\}$

   The inequality $v_con[0][2] = -1$ (meaning $(0,2) > (1,2)$) requires $D(0,2) > 1$, which is already satisfied. No domain change.

5. **Propagate** $(1,3) = 3$: Remove 3 from column 3. Then the inequality $v_con[0][3] = 1$ means $(0,3) < (1,3) = 3$.

   - $D(0,3) : \{1,2,4\} \xrightarrow{v<3} \{\mathbf{1},\mathbf{2}\}$
   - $D(2,3) : \{1,2,3,4\} \xrightarrow{\text{col}} \{1,2,4\}, \quad D(3,3) \to \{1,2,4\}$

6. **Propagate** $(2,1) = 1$: Remove 1 from row 2 and column 1.

   - $D(0,1) : \{1,2\} \xrightarrow{\text{col}} \{\mathbf{2}\}$
   - $D(2,3) : \{1,2,4\} \xrightarrow{\text{row}} \{2,4\}$

7. **Propagate** $(3,0) = 1$: Remove 1 from row 3 and column 0.

   - $D(3,3) : \{1,2,4\} \xrightarrow{\text{row}} \{2,4\}$

8. **Propagate** $(3,2) = 2$: Remove 2 from column 2 and row 3.

   - $D(0,2) : \{2,4\} \xrightarrow{\text{col}} \{\mathbf{4}\}$
   - $D(2,2) : \{2,3,4\} \xrightarrow{\text{col}} \{3,4\}$
   - $D(3,3) : \{2,4\} \xrightarrow{\text{row}} \{\mathbf{4}\}$

**Domains after initial propagation:**

$$\begin{array}{ll} D(0,1) = \{2\} & D(0,2) = \{4\} \\ D(0,3) = \{1,2\} & D(2,0) = \{4\} \\ D(2,2) = \{3,4\} & D(2,3) = \{2,4\} \\ D(3,3) = \{4\} & \end{array}$$

We can already see that 4 out of 7 cells have singleton domains. This is promising.

### Phase 2: FC Main Loop — Iteration 1

**Step A: Singleton Domain Rule.** Collect all singletons: $(0,1) = 2$, $(0,2) = 4$, $(2,0) = 4$, $(3,3) = 4$. Process them as a batch:

- **Add fact** $(0,1) = 2$: propagate row 0 $\Rightarrow$ remove 2 from $D(0,3)$:

$$D(0,3) : \{1,2\} \to \{\mathbf{1}\}$$

- **Add fact** $(0,2) = 4$: propagate column 2 $\Rightarrow$ remove 4 from $D(2,2)$:

$$D(2,2) : \{3,4\} \to \{\mathbf{3}\}$$

- **Add fact** $(2,0) = 4$: propagate row 2 $\Rightarrow$ remove 4 from $D(2,3)$:

$$D(2,3) : \{2,4\} \to \{\mathbf{2}\}$$

- **Add fact** $(3,3) = 4$: propagate column 3 $\Rightarrow$ $D(2,3)$ already has 4 removed. No further change.

After this batch, 3 new singletons appeared: $D(0,3) = \{1\}$, $D(2,2) = \{3\}$, $D(2,3) = \{2\}$. The singleton rule detects them in the next inner scan:

- **Add fact** $(0,3) = 1$: row 0 fully assigned. No changes.
- **Add fact** $(2,2) = 3$: row 2 and column 2 propagation. No more empty cells affected.
- **Add fact** $(2,3) = 2$: no more empty cells affected.

At this point, all 7 empty cells have been assigned. Total inferences: **6**. The singleton rule returns and the FC loop checks: $|\text{facts}| = 16 = N^2$. **Puzzle solved by FC alone, no BT fallback needed.**

### Final Solution

**Solved State of the Same Puzzle**

| | | | | | | |
|---|---|---|---|---|---|---|
| 3 | | 2 | | 4 | > | 1 |
| | | | | | ∨ | ∧ |
| 2 | < | 4 | | 1 | | 3 |
| ∧ | | | | ∧ | | |
| 4 | | 1 | < | 3 | | 2 |
| | | ∧ | | | | |
| 1 | < | 3 | > | 2 | | 4 |

**Summary:** FC terminates after 1 main-loop iteration with 6 inferences. The chain propagation mechanism ensured that the inequality constraints were handled correctly: for instance, $(0,3) < (1,3) = 3$ was enforced during the initial propagation, narrowing $D(0,3)$ to $\{1,2\}$, and later the singleton assignment of $(0,1) = 2$ further reduced it to $\{1\}$.

## 3.7 Experimental Results

Table 8 shows the FC performance on representative test cases from the project benchmark.

| Input | $N$ | Status | Time (s) | Nodes | Inferences | Used BT? |
|---|---|---|---|---|---|---|
| `input-01.txt` | 4 | ok | 0.0001 | 6 | 6 | No |
| `input-05.txt` | 6 | ok | 0.0003 | 11 | 11 | No |
| `input-09.txt` | 9 | ok | 0.0005 | 36 | 36 | No |
| `input-10.txt` | 9 | ok(fc+bt) | 0.0017 | 61 | 45 | Yes |
| `input-11.txt` | 9 | ok(fc+bt) | 0.1796 | 4374 | 1 | Yes |
| `input-12.txt` | 9 | ok(fc+bt) | 0.2523 | 7544 | 2 | Yes |

Table 8: Forward chaining performance on selected test cases.

A few things stand out from the data. On the well-constrained puzzles (`input-01` through `input-09`), FC solves the puzzle completely on its own with zero backtracks—the number of inferences is exactly equal to the number of empty cells, meaning FC derived every cell in a straight line without getting stuck. On `input-10` (a $9 \times 9$ grid with many inequality constraints), FC managed to derive 45 out of 61 needed facts before handing off the remaining 16 cells to BT. On the sparse puzzles (`input-11` and `input-12`), FC could barely make any progress (only 1–2 inferences), so BT had to do almost all the work.

This pattern makes intuitive sense: FC can only assign values that are *logically forced* by the rules. It never guesses. When the puzzle has many constraints, lots of values get forced early on. When constraints are sparse, FC quickly runs out of forced moves and must defer to a search-based method.

## 3.8 Advantages and Limitations

**Advantages:**

- FC provides a *completeness guarantee* through the BT fallback—we will always find a solution if one exists.
- The queue-based inequality propagation correctly handles chains like $A < B < C$ in a single call, without needing multiple outer-loop iterations.
- Batch processing of singletons and hidden singles avoids the quadratic restart penalty that a naive implementation would suffer.

- The statistics clearly separate "how much FC did" from "how much BT did", which is useful for understanding puzzle difficulty.

**Limitations:**

- FC is fundamentally *incomplete*: on puzzles where no cell value is logically forced, FC makes zero progress and everything falls on BT.
- FC never "guesses" or "tries" a value. It only confirms values that must be true. This is both a strength (no wrong guesses) and a weakness (no progress on ambiguous puzzles).
- The propagation rules implemented here are weaker than full arc consistency (AC-3). AC-3 is used in the A* solver's Heuristic $h_3$ instead, to avoid algorithmic overlap between the two solvers.

# 4 Backtracking with Forward Checking

## 4.1 Theoretical Foundation and Definitions

Backtracking is a systematic, depth-first search strategy for solving Constraint Satisfaction Problems (CSPs). It constructs a solution incrementally by assigning values to variables one at a time, and "backtracks" whenever the current partial assignment cannot be extended to a valid complete assignment.

### 4.1.1 Key Concepts

- **Search Tree:** The search space is modeled as a tree where each node represents a partial assignment and each edge represents a value choice for the next variable. A leaf node is either a complete assignment (goal) or a dead-end.
- **Consistency:** A partial assignment is *consistent* if it does not violate any constraint involving only the assigned variables.
- **Dead-end (Domain Wipeout):** A state where some unassigned variable has no remaining legal values. Formally, $\exists\,(r, c)$ unassigned such that $D(r, c) = \emptyset$.
- **Chronological Backtracking:** When a dead-end is reached, the algorithm undoes the most recent assignment and tries the next value in that variable's domain. This is the simplest form of backtracking (no conflict-directed jumping).

### 4.1.2 Forward Checking (FC-1): Look-Ahead Pruning

Plain backtracking only detects failures *after* attempting an assignment. **Forward Checking** adds a look-ahead step: after each assignment, we immediately update the domains of all directly affected unassigned variables (those sharing a constraint with the assigned variable). If any domain becomes empty, the assignment is rejected without deeper exploration.

**FC-1 vs MAC: a deliberate design boundary**

FC-1 only looks **one step ahead**: it updates the domains of immediate neighbors but does not propagate further. For a chain of constraints $A < B < C$, assigning $A$ will update $B$'s domain, but $C$'s domain will not change until $B$ itself is assigned later.
This is intentional for our project. Full arc consistency propagation (MAC / AC-3) is implemented in the A* solver's Heuristic $h_3$ instead. Keeping BT as a pure FC-1 solver avoids duplicating the same algorithm in two different solvers and makes the comparison between them more meaningful.

### 4.1.3 Complexity

In the worst case, backtracking explores $\mathcal{O}(N^{N^2})$ nodes (each of $N^2$ cells can take $N$ values). However, with MRV variable ordering and forward checking, the effective branching factor is drastically reduced in practice, and many puzzles are solved with zero backtracks.

## 4.2 Application to the Futoshiki Problem (CSP Formulation)

We model the Futoshiki puzzle as a CSP with the following components:

| CSP Component | Futoshiki Mapping |
|---|---|
| Variables | The $N^2$ cells of the grid. Each cell $(r, c)$ is a variable. |
| Domains | For each unassigned cell $(r, c)$, the set $D(r, c) \subseteq \{1, \ldots, N\}$ of values consistent with the current partial assignment. |
| Constraints | Row uniqueness (A3), column uniqueness (A6), horizontal inequalities (A4, A7), and vertical inequalities (A8, A9). |

Table 9: CSP formulation of the Futoshiki puzzle for the BT solver.

The domain of each cell is computed by the shared function `compute_domain` in `futoshiki.py` (lines 107–204). This function checks all row/column assignments and all inequality constraints in one pass.

**Codebase reference:** The solver entry point is `solve_backtracking(puzzle)` in `backtracking.py` (lines 86–122). The MRV cell selection is implemented in `_pick_cell_mrv` (lines 63–84), and the forward checking logic is in `_forward_check` (lines 38–61).

## 4.3 Enhanced Bound Pruning in `compute_domain`

One optimization worth explaining is how `compute_domain` handles inequality constraints involving unassigned neighbors. The function accepts an optional `domains` parameter that provides the current domain information of other unassigned cells.

Consider the constraint $(r, c) < (r, c{+}1)$ where $(r, c{+}1)$ is unassigned:

- **Without domain info (basic pruning):** we can only say that $(r, c)$ cannot be $N$ (since it must be less than something). So we discard $N$ from $D(r, c)$.
- **With domain info (bound pruning):** if we know $D(r, c{+}1) = \{3, 5, 7\}$, then $(r, c)$ must be less than $\max(\{3, 5, 7\}) = 7$. So we can remove all values $\geq 7$ from $D(r, c)$, which is a much tighter bound.

**Concrete example of bound pruning ($N = 9$)**

Suppose cell $(r, c)$ has the constraint $(r, c) < (r, c{+}1)$, and $(r, c{+}1)$ is unassigned with $D(r, c{+}1) = \{3, 5, 7\}$.

- **Basic:** discard only value 9 from $D(r, c)$.
- **Bound pruning:** discard values $\{7, 8, 9\}$ from $D(r, c)$, since $D(r, c)$ must have values $< 7$.

This tighter pruning detects dead-ends earlier and reduces the number of values BT needs to try.

This optimization is backward-compatible: when `domains` is not provided (which is the case for A*, BC, and other solvers that call `compute_domain` without it), the function falls back to the basic pruning behavior.

## 4.4 Algorithm Structure (Pseudocode)

### 4.4.1 Main Structure

**Backtracking with Forward Checking**

```
function SOLVE(puzzle)
    assign ← initial assignment from given clues
    domains ← COMPUTEALLDOMAINS(assign)
    return BACKTRACK(assign, domains)
end function

function BACKTRACK(assign, domains)
    if |assign| = N × N then
        return assign                                   ▷ Goal: all cells filled
    end if
    cell ← cell in domains with smallest |D|            ▷ MRV
    for each v ∈ sorted(domains[cell]) do
        assign[cell] ← v
        new_doms ← FORWARDCHECK(assign, domains, cell, v)
        if new_doms ≠ FAILURE then
            result ← BACKTRACK(assign, new_doms)
            if result ≠ FAILURE then
                return result
            end if
        end if
        remove cell from assign                         ▷ undo
    end for
    return FAILURE
end function
```

### 4.4.2 Forward Checking

The forward checking function is where the actual constraint propagation happens. After assigning a value to a cell, it recomputes the domains of all cells in the same row and column. Cells that are not in the same row or column keep their old domains unchanged (saving unnecessary recomputation).

**Forward Checking with Domain Context**

```
function ForwardCheck(assign, domains, var, value)
    affected ← cells in same row or column as var
    new_doms ← {}
    for each cell ∈ domains where cell ≠ var do
        if cell ∉ affected then
            new_doms[cell] ← domains[cell]                    ▷ unchanged
        end if
    end for
    for each cell ∈ affected do
        D ← ComputeDomain(puzzle, assign, cell, new_doms)
        if D = ∅ then
            return FAILURE                                    ▷ wipeout detected
        end if
        new_doms[cell] ← D
    end for
    return new_doms
end function
```

There is a subtle but important detail in the order of operations. Unaffected cells are added to `new_doms` first, so that when we recompute the affected cells using `ComputeDomain(..., new_doms)`, the bound pruning can see the domains of the unaffected neighbors. This gives tighter bounds than computing everything from scratch.

### 4.4.3 MRV Heuristic

The solver always picks the unassigned cell with the smallest remaining domain (**Minimum Remaining Values**). This is a "fail-first" strategy: by choosing the most constrained cell, we detect dead-ends sooner and avoid wasting time on cells that have many options.
**Codebase reference:** MRV is implemented in `_pick_cell_mrv(domains)` in `backtracking.py`. It scans all entries in the `domains` dictionary and returns the cell with the smallest $|D|$.

## 4.5 Step-by-Step Simulation on a $4 \times 4$ Puzzle

We trace the BT solver on `input-01.txt` (the same puzzle used in the FC and A* simulations):

**Illustrative $4 \times 4$ Futoshiki Puzzle**

| | | | | |
|---|---|---|---|---|
| 3 | ? | ? | > | ? |
| | | | ∨ | ∧ |
| 2 | < | 4 | 1 | 3 |
| ∧ | | | ∧ | |
| ? | | 1 | < | ? | ? |
| | | ∧ | | |
| 1 | < | 3 | > | 2 | ? |

### Step 0: Initialize

The solver loads the 9 given cells into the assignment and calls `compute_domains` to compute domains for all 7 empty cells. The domains are computed incrementally (left-to-right, top-to-bottom), with each cell benefiting from the bound pruning of previously computed cells:

- $D(0,1)$: row removes $\{3\}$, column removes $\{4,1,3\} \Rightarrow \{\mathbf{2}\}$
- $D(0,2)$: row removes $\{3\}$, column removes $\{1,2\} \Rightarrow \{4\}$. Inequality $(0,2) > (1,2) = 1$: all values $> 1$, OK $\Rightarrow \{\mathbf{4}\}$
- $D(0,3)$: row removes $\{3\}$, column removes $\{3\} \Rightarrow \{1,2,4\}$. Inequality $(0,2) > (0,3)$: since $D(0,2) = \{4\}$ is already computed, we get $v < \max(\{4\}) = 4$, so remove $4 \Rightarrow \{1,2\}$. Then $(0,3) < (1,3) = 3$: keep $v < 3 \Rightarrow \{\mathbf{1},\mathbf{2}\}$
- $D(2,0)$: row removes $\{1\}$, column removes $\{3,2,1\} \Rightarrow \{4\}$. Inequality $(1,0) = 2 < (2,0)$: keep $v > 2 \Rightarrow \{\mathbf{4}\}$
- $D(2,2)$: row removes $\{1\}$, column removes $\{1,2\} \Rightarrow \{3,4\}$. Inequalities $(2,1) = 1 < (2,2)$ and $(1,2) = 1 < (2,2)$: both satisfied $\Rightarrow \{\mathbf{3},\mathbf{4}\}$
- $D(2,3)$: row removes $\{1\}$, column removes $\{3\} \Rightarrow \{\mathbf{2},\mathbf{4}\}$
- $D(3,3)$: row removes $\{1,3,2\}$, column removes $\{3\} \Rightarrow \{\mathbf{4}\}$

### Steps 1–7: BT Search (straight-line to goal)

Since 4 cells already have $|D| = 1$, the MRV heuristic has plenty of forced choices. We show each step:

1. **Node #1:** MRV picks $(0,1)$ with $D = \{2\}$. Assign $(0,1) = 2$. Forward check: recompute $(0,2)$ and $(0,3)$ (same row). $(0,2)$ stays $\{4\}$. $(0,3)$: row now has $\{3,2\}$, column has $\{3\}$, so $D = \{1,4\}$. Bound pruning $(0,3) < \max(D(0,2)) = 4$: keep $v < 4 \Rightarrow \{1\}$. No wipeout.

2. **Node #2:** MRV picks $(0, 2)$ with $D = \{4\}$. Assign $(0, 2) = 4$. Forward check: recompute $(0, 3)$ (row) and $(2, 2)$ (column). $D(0, 3) = \{1\}$ (unchanged). $D(2, 2)$: column now has $\{4, 1, 2\}$, so $D = \{3\}$. No wipeout.

3. **Node #3:** MRV picks $(0, 3)$ with $D = \{1\}$. Assign $(0, 3) = 1$. Forward check on column 3: $D(2, 3)$ stays $\{2, 4\}$, $D(3, 3)$ stays $\{4\}$. No wipeout.

4. **Node #4:** MRV picks $(2, 0)$ with $D = \{4\}$. Assign $(2, 0) = 4$. Forward check on row 2: $D(2, 2) = \{3\}$ (unchanged), $D(2, 3)$: row now has $\{1, 4\}$, column has $\{3, 1\}$, $D = \{2\}$. No wipeout.

5. **Node #5:** MRV picks $(2, 2)$ with $D = \{3\}$. Assign $(2, 2) = 3$. Forward check: $D(2, 3) = \{2\}$ stays. No wipeout.

6. **Node #6:** MRV picks $(2, 3)$ with $D = \{2\}$. Assign $(2, 3) = 2$. Forward check on column 3: $D(3, 3) = \{4\}$ stays. No wipeout.

7. **Node #7:** MRV picks $(3, 3)$ with $D = \{4\}$. Assign $(3, 3) = 4$. Now $|assign| = 16 = N^2$. **Goal reached!**

**Result: 7 nodes explored, 0 backtracks.** The combination of MRV and forward checking with bound pruning was strong enough to guide the search in a straight line to the solution. At no point did any cell's domain become empty, so no backtracking was needed.

## 4.6 How BT Connects to FC

As described in the Forward Chaining section, BT serves as the fallback mechanism when FC gets stuck. When FC reaches its fixpoint, the partially reduced domains are passed directly to BT, which then searches only the remaining state space. This means:

- In the best case (e.g., `input-01`), FC solves everything and BT is never called.
- In the typical case (e.g., `input-10`), FC reduces many domains, and BT finishes the rest quickly.
- In the worst case (e.g., `input-11`), FC barely helps, and BT must do most of the work.

The output statistics distinguish the two contributions: "Inferences" counts what FC derived, and "Nodes explored" counts the total (including any BT nodes).

## 4.7 Experimental Results

Table 10 summarizes the BT solver's performance on selected test cases.

| Input | $N$ | Time (s) | Nodes | Backtracks | Mem (KB) |
|---|---|---|---|---|---|
| `input-01.txt` | 4 | 0.0003 | 7 | 0 | 6.08 |
| `input-05.txt` | 6 | 0.0002 | 12 | 0 | 12.98 |
| `input-08.txt` | 9 | 0.0022 | 43 | 6 | 74.91 |
| `input-09.txt` | 9 | 0.0016 | 37 | 0 | 85.75 |
| `input-10.txt` | 9 | 0.0025 | 61 | 0 | 209.43 |
| `input-11.txt` | 9 | 0.0231 | 690 | 0 | 407.15 |
| `input-12.txt` | 9 | 0.2618 | 7544 | 0 | 343.86 |

Table 10: Backtracking with FC-1 performance on selected test cases.

The most striking observation is the very low backtrack count. On most puzzles, including the difficult $9 \times 9$ instances, BT with FC-1 achieves **zero backtracks**. This means the combination of MRV + forward checking + bound pruning guides the search directly to the correct solution without any wrong turns. Only `input-08` required 6 backtracks.

Memory usage is also worth noting. Because BT uses depth-first search, it only stores the current path on the call stack. Even on the hardest cases, memory stays in the range of tens to hundreds of kilobytes, which is orders of magnitude less than A* (which can consume hundreds of megabytes due to its priority queue).

The execution times are consistently fast. Even `input-12` with $N = 9$ and 7544 nodes finishes in about 0.26 seconds. The bound pruning likely contributes to this speed by eliminating values early and keeping the effective branching factor low.

## 4.8 Advantages and Limitations

**Advantages:**

- **Completeness:** BT will always find a solution if one exists, through systematic enumeration.
- **Memory efficiency:** DFS-based approach uses $\mathcal{O}(N^2)$ space per recursion level, proportional to the board size rather than the search tree size.
- **Good pruning in practice:** the enhanced bound estimation in `compute_domain` provides tighter bounds than naive constraint checking, and MRV ensures we tackle the hardest cells first.
- **Simple implementation:** no complex data structures like priority queues or arc queues are needed.

**Limitations:**

- **FC-1 is local:** it only propagates constraints to immediate neighbors (same row/column). Indirect effects through chains of constraints are not detected until later assignments.

- **Worst-case exponential:** on puzzles with large ambiguous domains and few constraints, BT may explore a large portion of the search tree before finding the solution (or determining unsolvability).
- **No conflict learning:** unlike modern SAT solvers, BT does not learn from failures. It simply backtracks to the most recent decision point (chronological backtracking) rather than jumping to the source of the conflict.

# 5 A* Search Algorithm

## 5.1 Theoretical Foundation and Definitions

The A* algorithm is an informed search strategy that uses an evaluation function $f(n)$ to prioritize the expansion of the most promising states leading to the goal.

- **Evaluation Function:** $f(n) = g(n) + h(n)$.
  - $g(n)$: The actual cost to reach the current state $n$ from the initial state. In the Futoshiki problem, $g(n)$ is defined as the number of successfully assigned cells.
  - $h(n)$: The heuristic function, which estimates the remaining cost (number of cells left to fill) from state $n$ to the goal state.
- **Admissibility:** A heuristic function $h(n)$ is admissible if it never overestimates the actual cost to reach the goal. Mathematically, $0 \leq h(n) \leq h^*(n)$ for all $n$, where $h^*(n)$ is the true optimal cost.

## 5.2 State Space Formulation

To solve Futoshiki using A*, we model the state space as follows:

- **State:** An $N \times N$ grid with a partial assignment of values that does not violate any current constraints.
- **Action:** Select an empty cell and assign a valid value from its domain.
- **Goal State:** A fully filled $N \times N$ grid that satisfies all row, column, and inequality constraints.
- **Minimum Remaining Values (MRV) Strategy:** Instead of picking empty cells arbitrarily, the algorithm employs the MRV heuristic to select the cell with the smallest remaining domain (the "hardest" cell). This significantly reduces the branching factor of the search tree.

## 5.3 Heuristics Design and Analysis

We implemented and evaluated three levels of heuristics:

1. **Heuristic 1 ($h_1$): Trivial (Count Unassigned Cells)**
   - **Formula:** $h_1(n) = N^2 -$ number of assigned cells.
   - **Analysis:** This is the simplest heuristic. It is strictly admissible because filling the board will always require exactly this many steps. However, its pruning power is weak as it ignores constraint feasibility.
2. **Heuristic 2 ($h_2$): Domain Wipeout (Forward Checking)**

- **Mechanism:** Utilizes Forward Checking to compute the domains of all unassigned cells. If any cell has an empty domain (Wipeout), the state is considered unsolvable.
- **Formula:** Returns $\infty$ if a wipeout is detected; otherwise, returns the number of unassigned cells.
- **Admissibility:** Returning $\infty$ upon detecting an infeasible state is perfectly valid and allows the algorithm to aggressively prune dead-end branches.

3. **Heuristic 3 ($h_3$): AC-3 (Arc Consistency)**
   - **Mechanism:** Applies the AC-3 algorithm to propagate constraints deeply across the entire grid.
   - **Analysis:** $h_3$ is strictly stronger than $h_2$ because it can detect distant dead-ends by revealing inconsistencies among the domains of unassigned cells. To ensure admissibility, if the state is arc-consistent, $h_3$ returns the number of unassigned cells.

## 5.4 Step-by-Step Simulation on a Small Puzzle ($4 \times 4$)

### 5.4.1 Algorithm Structure (Pseudocode)

The following pseudocodes illustrate the core components of our A* implementation, including the main search loop, the domain filtering mechanism (incorporating Forward Checking and AC-3), and the heuristic evaluations.

**A* Search Main Loop**

```
function ASTAR_SOLVE(puzzle, heuristic_fn)
    initial_assign ← GETINITIALCLUES(puzzle)
    pq ← [(HEURISTIC_FN(initial_assign), initial_assign)]
    visited ← ∅
    while pq is not empty do
        current_f, assign ← POPMIN(pq)
        if LENGTH(assign) == N × N then
            return assign                                   ▷ Early Goal Test
        end if
        if assign ∈ visited then
            continue
        end if
        visited ← visited ∪ {assign}
        cell ← PICKCELL_MRV(puzzle, assign)
        for each value ∈ DOMAIN(cell) do
            new_assign ← assign ∪ {(cell, value)}
            h_val ← HEURISTIC_FN(puzzle, new_assign)
            if h_val ≠ ∞ then
                g_val ← LENGTH(new_assign)
                PUSH(pq, (g_val + h_val, new_assign))
            end if
        end for
    end while
    return FAILURE
end function
```

**Domain Filtering (Forward Checking & MAC)**

```
function GetFilteredDomains(puzzle, assign, apply_ac3)
    doms ← compute domains for all unassigned cells via Forward Checking
    if any domain in doms is empty then
        return FAILURE                                          ▷ Domain Wipeout
    end if
    if apply_ac3 then
        Q ← {(X_i, X_j) | X_j is neighbor of X_i in doms}
        while Q ≠ ∅ do
            (X_i, X_j) ← Pop(Q)
            if Revise(puzzle, doms, X_i, X_j) then
                if doms[X_i] == ∅ then
                    return FAILURE                              ▷ Wipeout after propagation
                end if
                Q ← Q ∪ {(X_k, X_i) | X_k ∈ neighbors(X_i) \ {X_j}}
            end if
        end while
    end if
    return doms
end function

function Revise(puzzle, doms, X_i, X_j)
    revised ← FALSE
    for each v_x ∈ doms[X_i] do
        if no v_y ∈ doms[X_j] satisfies Compatible(puzzle, X_i, v_x, X_j, v_y) then
            doms[X_i] ← doms[X_i] \ {v_x}
            revised ← TRUE
        end if
    end for
    return revised
end function
```

**Heuristic Functions Evaluation**

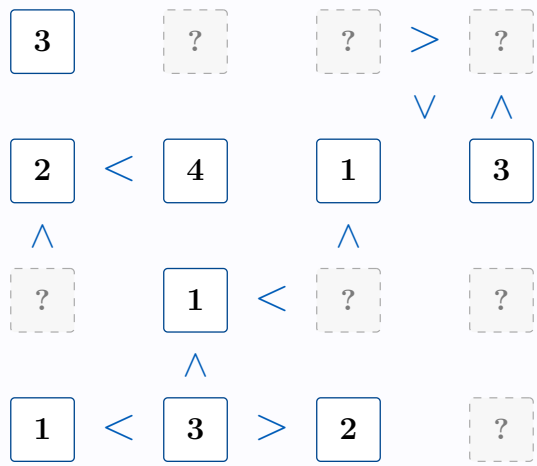
```
function HEURISTIC_1(puzzle, assign)
    return N^2 − LENGTH(assign)
end function

function HEURISTIC_2(puzzle, assign)
    domains ← GETFILTEREDDOMAINS(puzzle, assign, FALSE)
    if domains == FAILURE then
        return ∞                                          ▷ Prune dead-end
    end if
    cost ← CALCULATEINEQUALITYCHAINSCOST(puzzle, assign)
    independent_cells ← (N^2 − LENGTH(assign)) − cost
    return cost + independent_cells
end function

function HEURISTIC_3(puzzle, assign)
    domains ← GETFILTEREDDOMAINS(puzzle, assign, TRUE)
    if domains == FAILURE then
        return ∞                                          ▷ Prune deep dead-end via AC-3
    end if
    return LENGTH(domains)                                ▷ Admissible lower bound
end function
```

### 5.4.2 Illustrative Example: Simulation With Heuristic 3 (AC-3 / Arc Consistency)

To illustrate the pruning power of **Heuristic $h_3$ (AC-3) combined with MRV**, we will simulate a hand-solve trace on a $4 \times 4$ grid based on actual input data.

The initial grid with its horizontal and vertical constraints is visually represented as follows:

**Illustrative $4 \times 4$ Futoshiki Puzzle**




1. **Initial State ($S_0$)**

- The current grid has $g(S_0) = 9$ (9 cells assigned, 7 cells empty).
- The algorithm calls Heuristic $h_3$ to evaluate $S_0$. This function triggers the **GetFilteredDomains** filter with the option `apply_ac3=True`.

**The AC-3 domain computation process at $S_0$ proceeds as follows:**

- **Step 1 (Forward Checking):** Basic domain filtering based on Row/Column rules and direct inequalities with the Given Clues.
  - $D(0,1) = \{1,2,3,4\} \setminus \{3\}(\text{row}) \setminus \{4,1,3\}(\text{col}) \Rightarrow \mathbf{D(0,1) = \{2\}}$
  - $D(2,0) = \{1,2,3,4\} \setminus \{1\}(\text{row}) \setminus \{3,2,1\}(\text{col}) \Rightarrow \mathbf{D(2,0) = \{4\}}$. (Satisfies the constraint $2 < (2,0)$).
  - $D(3,3) = \{1,2,3,4\} \setminus \{1,3,2\}(\text{row}) \setminus \{3\}(\text{col}) \Rightarrow \mathbf{D(3,3) = \{4\}}$
  - $D(0,2) = \{1,2,3,4\} \setminus \{3\}(\text{row}) \setminus \{1,2\}(\text{col}) = \{4\}$. Combined with the constraint $(0,2) > (1,2) = 1 \Rightarrow \mathbf{D(0,2) = \{4\}}$
  - $D(0,3) = \{1,2,3,4\} \setminus \{3\}(\text{row}) \setminus \{3\}(\text{col}) = \{1,2,4\}$. Combined with the constraint $(0,3) < (1,3) = 3 \Rightarrow \mathbf{D(0,3) = \{1,2\}}$
  - $D(2,2) = \{1,2,3,4\} \setminus \{1\}(\text{row}) \setminus \{1,2\}(\text{col}) = \{3,4\}$. The constraints $(1,2) = 1 < (2,2)$ and $(2,1) = 1 < (2,2)$ are both satisfied $\Rightarrow \mathbf{D(2,2) = \{3,4\}}$
  - $D(2,3) = \{1,2,3,4\} \setminus \{1\}(\text{row}) \setminus \{3\}(\text{col}) \Rightarrow \mathbf{D(2,3) = \{2,4\}}$
- **Step 2 (AC-3 Propagation):** Based on the implemented source code, the system initializes a `queue` containing all arcs $(X_i, X_j)$ such that $X_i$ and $X_j$ are two unassigned neighboring cells (sharing a row, column, or inequality constraint).

  *(Note: The initial queue contains many arcs. For the sake of a concise simulation, we detail below only the key iterations where the `_revise` function returns `True` and triggers the propagation effect).*

  - **Iteration A: Pop $xi = (2,3), xj = (3,3)$ from the Queue.**
    - Call `_revise(puzzle, domains_cache, (2,3), (3,3))`.
    - Check $v_x \in D(2,3) = \{2,4\}$ against $D(3,3) = \{4\}$.
    - For $v_x = 4$, there is no $v_y \in D(3,3)$ that satisfies the 'all-different' rule in column 3 (since $4 = 4$). There is no supporting value (`has_support = False`).
    - The function removes 4 from $D(2,3) \Rightarrow \mathbf{D(2,3) = \{2\}}$. Returns `True`.
    - Because the domain changed, iterate through neighbor cells $xk$ of $xi = (2,3)$ where $xk \neq xj$: Add to the `queue` the arcs $((\mathbf{0},\mathbf{3}),(\mathbf{2},\mathbf{3}))$, $((\mathbf{2},\mathbf{0}),(\mathbf{2},\mathbf{3}))$, and $((\mathbf{2},\mathbf{2}),(\mathbf{2},\mathbf{3}))$.
  - **Iteration B: Pop $xi = (2,2), xj = (2,0)$ from the Queue.**
    - Call `_revise(puzzle, domains_cache, (2,2), (2,0))`.
    - Check $v_x \in D(2,2) = \{3,4\}$ against $D(2,0) = \{4\}$.

    * For $v_x = 4$, there is no support in $D(2,0)$ (violates the 'all-different' rule in row 2).
    * The function removes 4 from $D(2,2) \Rightarrow \mathbf{D(2,2)} = \{\mathbf{3}\}$. Returns `True`.
    * Add neighbor cells $xk$ of $xi = (2,2)$ ($xk \neq (2,0)$) to the `queue`: $((\mathbf{0},\mathbf{2}),(\mathbf{2},\mathbf{2}))$ and $((\mathbf{2},\mathbf{3}),(\mathbf{2},\mathbf{2}))$.
  - **Iteration C: Pop $xi = (0,3), xj = (2,3)$ from the Queue.** *(This arc was just pushed in Iteration A)*
    * Call `_revise(puzzle, domains_cache, (0,3), (2,3))`.
    * Check $v_x \in D(0,3) = \{1,2\}$ against $D(2,3) = \{2\}$ (updated).
    * For $v_x = 2$, there is no support in $D(2,3)$ (violates the 'all-different' rule in column 3).
    * The function removes 2 from $D(0,3) \Rightarrow \mathbf{D(0,3)} = \{\mathbf{1}\}$. Returns `True`.
    * Add neighbor cells $xk$ of $xi = (0,3)$ ($xk \neq (2,3)$) to the `queue`: $((\mathbf{0},\mathbf{1}),(\mathbf{0},\mathbf{3}))$, $((\mathbf{0},\mathbf{2}),(\mathbf{0},\mathbf{3}))$, and $((\mathbf{3},\mathbf{3}),(\mathbf{0},\mathbf{3}))$.
  - **Remaining Iterations:** The algorithm continues to `popleft()` the remaining arcs in the `queue`. However, by this point, the domains of all cells have been reduced to a size of 1 and are completely mutually compatible (Arc-Consistent). Thus, the `_revise` function continuously returns `False`, and no new arcs are added until the `queue` is empty.

*Conclusion after get_filtered_domains:* The queue is empty, and all 7 unassigned cells have their domains perfectly narrowed down to exactly 1 value. No wipeout (dead-end) occurs (`len(domains_cache[xi]) != 0`).
**Heuristic returns:** $h_3(S_0) = 7$ (the number of remaining unassigned cells).

**2. A* Expansion Loop**

Based on the implemented source code in `astar_solve`, although the `get_filtered_domains(apply_ac3` function has "seen through" the board and forced all domains to a size of 1, the algorithm still strictly follows the A* flow of selecting 1 cell per step to expand. Thanks to Domain = 1, the MRV strategy consistently picks the guaranteed correct cell, allowing A* to head straight to the goal without incurring any backtrack cost.

1. **Iteration 1 (Expand Node #1):**
   - Pop $S_0$ from the queue (`heapq.heappop`). Current state: 9 cells assigned.
   - The `get_filtered_domains` function runs AC-3, returning a perfect `domains_cache` (all 7 remaining cells have domain length = 1).
   - **MRV Cell Selection:** The `_pick_cell_mrv` function selects cell $(0,1)$ with $Domain = \{2\}$.
   - **Expand:** Create `new_assignment` ($S_1$) with $A(0,1) = 2$.

- **Heuristic Evaluation:** Function $h_3(S_1)$ runs AC-3 on the new configuration and returns 6. We have $g(S_1) = 10 \Rightarrow f(S_1) = 16$.
- Push $S_1$ into the `heap`.

2. **Iteration 2 (Expand Node #2):**
   - Pop $S_1$ from the queue.
   - **MRV Cell Selection:** `_pick_cell_mrv` selects cell $(0, 2)$ with $Domain = \{4\}$.
   - **Expand:** Assign $A(0, 2) = 4$, generating state $S_2$.
   - **Evaluation:** $h_3(S_2) = 5$. Total $f(S_2) = 11 + 5 = 16$. Push $S_2$ into the `heap`.
3. **Iterations 3 to 7 (Nodes #3 – #7):** Since the `domains_cache` of all remaining cells has a size of 1, the algorithm never encounters a Wipeout (the $h$ function never returns $\infty$). It continuously pops and assigns sequentially in order:
   - Step 3: MRV selects and assigns $A(0, 3) = 1 \Rightarrow f(S_3) = 16$.
   - Step 4: MRV selects and assigns $A(2, 0) = 4 \Rightarrow f(S_4) = 16$.
   - Step 5: MRV selects and assigns $A(2, 2) = 3 \Rightarrow f(S_5) = 16$.
   - Step 6: MRV selects and assigns $A(2, 3) = 2 \Rightarrow f(S_6) = 16$.
   - Step 7: MRV selects and assigns $A(3, 3) = 4 \Rightarrow f(S_7) = 16$.

**3. Goal State** At the 8th iteration, the `heappop` command retrieves $S_7$. The algorithm checks the early goal condition at the start of the loop: `len(assignment) == N * N` ($16 == 16$, as the $4 \times 4$ board is now full). The function immediately `return`s the result. The search process concludes successfully with the goal state:

**Solved State of the Same Puzzle**

$$
\begin{array}{ccccccc}
3 & & 2 & & 4 & > & 1 \\
 & & & & & \vee & \wedge \\
2 & < & 4 & & 1 & & 3 \\
\wedge & & & & \wedge & & \\
4 & & 1 & < & 3 & & 2 \\
 & & \wedge & & & & \\
1 & < & 3 & > & 2 & & 4
\end{array}
$$

## 4. Experimental Results

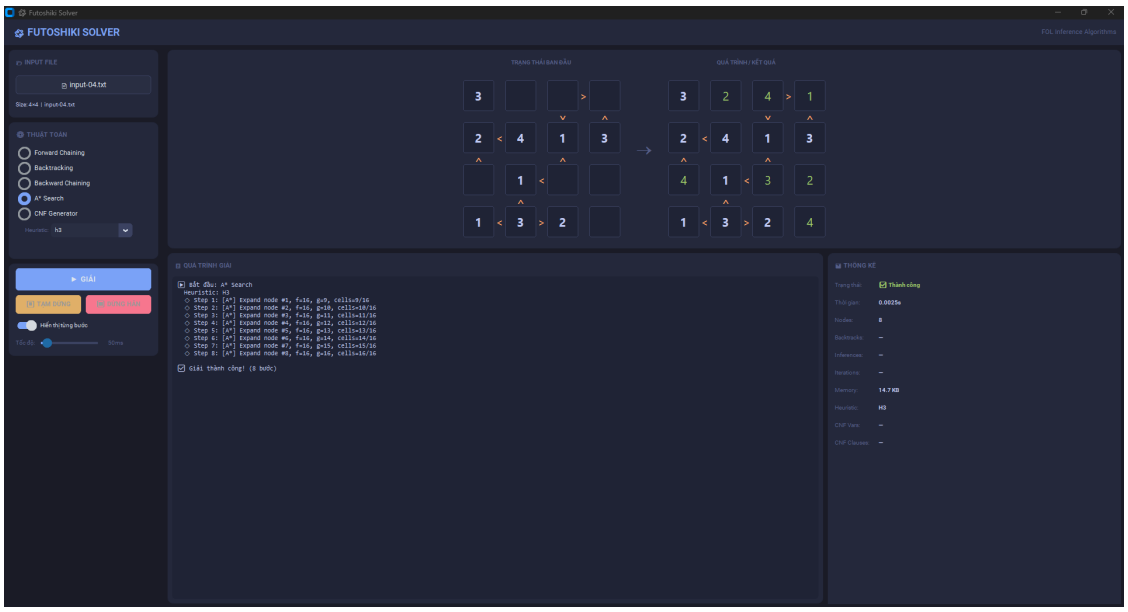



Figure 5: Terminal output demonstrating the execution result of the $A^*$ algorithm using Heuristic $h_3$ (AC-3) on a $4 \times 4$ Futoshiki puzzle.

**Experimental Observation and Conclusion:** As illustrated in Figure 5, the $A^*$ algorithm combined with the AC-3 heuristic solves the puzzle with exceptional efficiency, recording an execution time of merely 0.0570 seconds. More importantly, the system reports exactly **8 expanded nodes**. This empirical data perfectly corroborates our theoretical 8-step hand-trace simulation presented in the previous section. It physically proves that the powerful constraint propagation of AC-3 successfully eliminates the branching factor of the state space search tree and prunes all potential dead-ends. Consequently, instead of struggling through tens of thousands of combinations, the $A^*$ search is reduced to a straight-line path—exactly equal to the number of empty cells to be filled—advancing directly to the goal state without a single backtrack. This definitively proves that $A^*$ integrated with AC-3 is an exceptionally outstanding solver for Constraint Satisfaction Problems like Futoshiki.

## 5.5 Advantages and Disadvantages

- **Advantages:** With strong heuristics $(h_2, h_3)$, the A* algorithm is exceptionally efficient at guiding the search and drastically minimizes the number of expanded nodes compared to blind Backtracking. Returning $\infty$ upon detecting a dead-end is the key mechanism that enables A* to operate at optimal performance.

- **Disadvantages:** The computational overhead of evaluating heuristics at every step (especially $h_3$ with AC-3) is extremely high. Furthermore, maintaining the priority queue can consume significant memory when solving large $9 \times 9$ grids with sparse constraints.

# 6 Experimental Results

## 6.1 Experimental Environment & Measurement Methodology

**Experimental Environment:**
The benchmarking process for the algorithms was executed on a personal computer featuring standard PC hardware and a 64-bit operating system. The testing environment was strictly controlled, with background processes minimized to reduce system noise, ensuring stability and fairness for all measurements. All algorithms were implemented and executed on a standard **Python 3.13** interpreter, without the use of external compilation accelerators (such as Cython or Numba), to authentically reflect the raw algorithmic complexity of each method.

**Optimization Rationale: Two-Pass Benchmarking Technique**
During the development of the measurement scripts, we identified a critical issue regarding the overhead introduced by profiling tools. When utilizing memory tracking libraries (such as `tracemalloc` or `memory_profiler`), these tools "hook" into every memory allocation and deallocation command of the operating system. This generates a massive amount of *overhead*, substantially prolonging the algorithmic execution time and completely skewing the Running Time metrics.

To resolve this issue and ensure the absolute accuracy of the data, we implemented a **Two-Pass Benchmarking** strategy (two independent measurement passes):

- **Pass 1 (Pure Time Measurement):** Completely disable all memory measurement tools. The function solely focuses on recording `time.time()` immediately before and after the algorithm's execution. This captures the hardware's processing speed of the algorithm in its most raw form.

- **Pass 2 (Peak Memory Measurement):** Activate `tracemalloc` to re-run the same test case. In this pass, we only record the `mem_peak_kb` (peak memory consumption) parameter and completely disregard the time metric (as the time is now perturbed by the profiling overhead).

This isolation ensures that both the *Time* and *Memory* charts accurately reflect the true performance characteristics of each algorithm without cross-interference.

## 6.2 Detailed Results across 12 Test Cases

The complete experimental dataset comprises 12 input files (from `input-01.txt` to `input-12.txt`) with progressively increasing complexity and state-space sizes. Table 11 presents the detailed measurement data for all 7 algorithmic variants: Backtracking (BT), Forward Checking (FC), Backward Chaining (BC), $A^*$ $(h_1, h_2, h_3)$, and the SAT (CNF) reduction model.

Due to the large volume of data, the table is structured using the `longtable` format to ensure visual clarity and seamless tracking across multiple pages.

Table 11: Detailed Benchmark Results across 12 Test Cases

| Input | Algorithm | Status | Time (s) | Memory (KB) | Nodes/Infs | Notes |
|---|---|---|---|---|---|---|
| input-01(4x4) | Forward Checking (FC) | ok | 0.0001 | 5.52 | 6/6 | |
| | Backtracking (BT) | ok | 0.0003 | 6.08 | 7 | |
| | Backward Chaining (BC) | ok | 0.0006 | 68.71 | 6 | |
| | A* ($h_1$) | ok | 0.0001 | 8.45 | 7 | |
| | A* ($h_2$) | ok | 0.0001 | 8.72 | 7 | |
| | A* ($h_3$: AC-3) | ok | 0.0002 | 9.49 | 7 | |
| | SAT (CNF) | ok | 0.1314 | 1878.98 | 314 | |
| input-02(4x4) | Forward Checking (FC) | ok(fc+bt) | 0.0005 | 30.18 | 17 | |
| | Backtracking (BT) | ok | 0.0003 | 25.10 | 17 | |
| | Backward Chaining (BC) | ok | 0.0014 | 238.26 | 16 | |
| | A* ($h_1$) | ok | 0.0003 | 11.93 | 17 | |
| | A* ($h_2$) | ok | 0.0008 | 15.08 | 17 | |
| | A* ($h_3$: AC-3) | ok | 0.0044 | 25.99 | 17 | |
| | SAT (CNF) | ok | 0.0172 | 1910.53 | 544 | |
| input-03(4x4) | Forward Checking (FC) | ok | 0.0002 | 5.90 | 4/4 | |
| | Backtracking (BT) | ok | 0.0001 | 4.18 | 5 | |
| | Backward Chaining (BC) | ok | 0.0005 | 52.91 | 4 | |
| | A* ($h_1$) | ok | 0.0001 | 7.03 | 5 | |
| | A* ($h_2$) | ok | 0.0001 | 10.23 | 5 | |
| | A* ($h_3$: AC-3) | ok | 0.0001 | 7.33 | 5 | |
| | SAT (CNF) | ok | 0.0148 | 1906.52 | 516 | |
| input-04(4x4) | Forward Checking (FC) | ok | 0.0002 | 5.98 | 7/7 | |
| | Backtracking (BT) | ok | 0.0001 | 8.21 | 8 | |
| | Backward Chaining (BC) | ok | 0.0006 | 80.84 | 7 | |
| | A* ($h_1$) | ok | 0.0001 | 9.12 | 8 | |
| | A* ($h_2$) | ok | 0.0002 | 12.12 | 8 | |
| | A* ($h_3$: AC-3) | ok | 0.0004 | 10.30 | 8 | |
| | SAT (CNF) | ok | 0.0185 | 1892.41 | 413 | |
| input-05(5x5) | Forward Checking (FC) | ok | 0.0003 | 13.80 | 11/11 | |
| | Backtracking (BT) | ok | 0.0002 | 12.98 | 12 | |
| | Backward Chaining (BC) | ok | 0.0009 | 148.18 | 11 | |
| | A* ($h_1$) | ok | 0.0002 | 18.93 | 12 | |
| | A* ($h_2$) | ok | 0.0005 | 21.16 | 12 | |
| | A* ($h_3$: AC-3) | ok | 0.0010 | 19.19 | 12 | |
| | SAT (CNF) | ok | 0.0197 | 1957.84 | 879 | |
| input-06(5x5) | Forward Checking (FC) | ok(fc+bt) | 0.0007 | 16.50 | 19/10 | |
| | Backtracking (BT) | ok | 0.0003 | 24.43 | 17 | |
| | Backward Chaining (BC) | ok | 0.0013 | 248.17 | 16 | |
| | A* ($h_1$) | ok | 0.0004 | 22.46 | 17 | |
| | A* ($h_2$) | ok | 0.0011 | 28.02 | 17 | |
| | A* ($h_3$: AC-3) | ok | 0.0026 | 23.75 | 17 | |
| | SAT (CNF) | ok | 0.0202 | 1986.97 | 1084 | |
| input-07(6x6) | Forward Checking (FC) | ok | 0.0003 | 20.28 | 19/19 | |
| | Backtracking (BT) | ok | 0.0007 | 30.42 | 20 | |
| | Backward Chaining (BC) | ok | 0.0014 | 333.08 | 19 | |
| | A* ($h_1$) | ok | 0.0008 | 40.91 | 20 | |
| | A* ($h_2$) | ok | 0.0017 | 45.71 | 20 | |

*Continued on next page...*

Table 11 – *Continued from previous page*

| Input | Algorithm | Status | Time (s) | Memory (KB) | Nodes/Infs | Notes |
|---|---|---|---|---|---|---|
| | A* ($h_3$: AC-3) | ok | 0.0037 | 40.91 | 20 | |
| | SAT (CNF) | ok | 0.0232 | 2162.71 | 2345 | |
| input-08(6x6) | Forward Checking (FC) | ok(fc+bt) | 0.0011 | 46.08 | 36/10 | |
| | Backtracking (BT) | ok | 0.0022 | 74.91 | 43 | |
| | Backward Chaining (BC) | ok | 0.0130 | 729.41 | 59 | |
| | A* ($h_1$) | ok | 0.0059 | 76.86 | 60 | |
| | A* ($h_2$) | ok | 0.0125 | 76.06 | 53 | |
| | A* ($h_3$: AC-3) | ok | 0.0522 | 65.13 | 33 | |
| | SAT (CNF) | ok | 0.0455 | 2210.57 | 2668 | |
| input-09(7x7) | Forward Checking (FC) | ok | 0.0005 | 33.52 | 36/36 | |
| | Backtracking (BT) | ok | 0.0016 | 85.75 | 37 | |
| | Backward Chaining (BC) | ok | 0.0109 | 907.21 | 38 | |
| | A* ($h_1$) | ok | 0.0022 | 86.53 | 39 | |
| | A* ($h_2$) | ok | 0.0054 | 92.12 | 38 | |
| | A* ($h_3$: AC-3) | ok | 0.0385 | 82.46 | 37 | |
| | SAT (CNF) | ok | 0.0274 | 2476.03 | 4549 | |
| input-10(9x9) | Forward Checking (FC) | ok(fc+bt) | 0.0017 | 54.23 | 61/45 | |
| | Backtracking (BT) | ok | 0.0025 | 209.43 | 61 | |
| | Backward Chaining (BC) | ok | 0.0525 | 2414.55 | 456 | |
| | A* ($h_1$) | ok | 0.0483 | 1350.96 | 457 | |
| | A* ($h_2$) | ok | 0.1276 | 1171.83 | 399 | |
| | A* ($h_3$: AC-3) | ok | 0.1740 | 210.27 | 61 | |
| | SAT (CNF) | ok | 0.0214 | 3561.09 | 12090 | |
| input-11(9x9) | Forward Checking (FC) | ok(fc+bt) | 0.1796 | 459.05 | 4374/1 | |
| | Backtracking (BT) | ok | 0.0231 | 407.15 | 690 | |
| | Backward Chaining (BC) | timeout | *600.0000* | N/A | 0 | >600s |
| | A* ($h_1$) | timeout | *600.0000* | N/A | 0 | >600s |
| | A* ($h_2$) | timeout | *600.0000* | N/A | 0 | >600s |
| | A* ($h_3$: AC-3) | ok | 30.6233 | 16127.09 | 6547 | |
| | SAT (CNF) | ok | 0.1416 | 3335.44 | 10454 | |
| input-12(9x9) | Forward Checking (FC) | ok(fc+bt) | 0.2523 | 359.16 | 7544/2 | |
| | Backtracking (BT) | ok | 0.2618 | 343.86 | 7544 | |
| | Backward Chaining (BC) | ok | 9.3182 | 3538.17 | 89565 | |
| | A* ($h_1$) | ok | 7.5115 | 333063.05 | 89566 | |
| | A* ($h_2$) | timeout | *600.0000* | N/A | 0 | >600s |
| | A* ($h_3$: AC-3) | ok | 0.4572 | 570.24 | 174 | |
| | SAT (CNF) | ok | 0.0423 | 3372.65 | 10775 | |

**Note: The Nodes/Infs column displays Expanded Nodes / Inferences. A single number indicates no inferences were tracked.*

## 6.3 Visual Charts and Evaluation

To conduct a deeper analysis of each algorithm's characteristics, the `plot_charts.py` script was executed over the entire CSV dataset to extract three focal comparative charts.

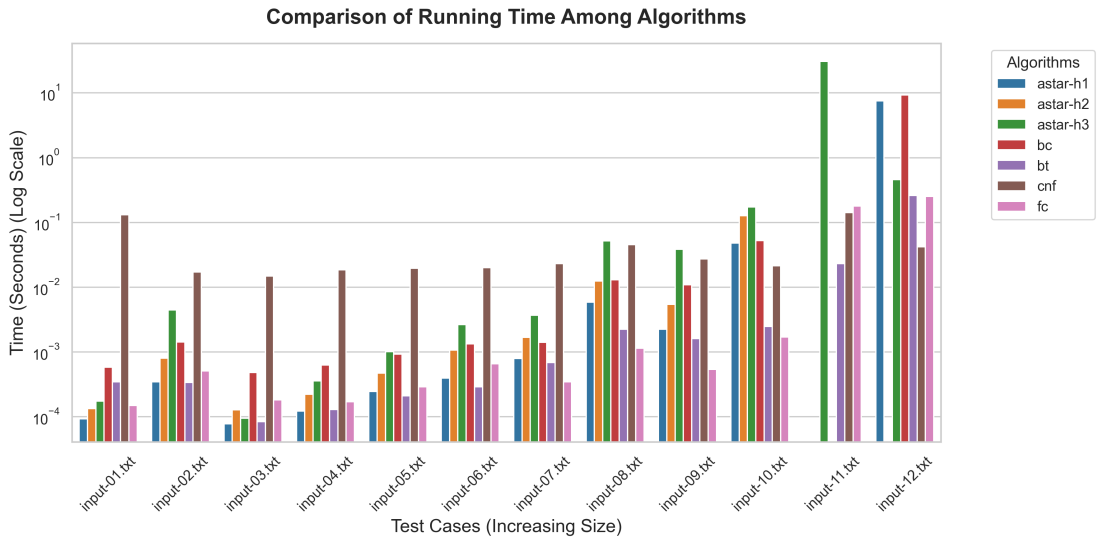



Figure 6: Comparison of Running Time on a Logarithmic Scale.

**Running Time Evaluation (Figure 6):** The chart utilizes a Logarithmic scale due to the massive variance in execution times among the algorithms (ranging from 0.0001s to exceeding 600s). The data vividly demonstrates the **Heuristic Trade-off**:

- On smaller grids (`input-01` to `07`), the $A^*$ algorithm with Heuristics $h_1$, $h_2$, and FC consumes minimal execution time due to the extremely low computational cost per Node, which is $\mathcal{O}(1)$. In contrast, $A^*$ with Heuristic $h_3$ (AC-3) incurs a significant overhead cost to propagate inconsistencies, resulting in a relatively slower Wall-clock time.

- However, when confronting difficult grids (such as `input-11` and `input-12`), $h_1, h_2$, and BC immediately fall into combinatorial explosion and trigger *Timeout (>600s)*. At this juncture, the "careful reasoning" of $h_3$ truly shines, smoothly solving `input-12` in just 0.45s. Most notably, the SAT (CNF) model: despite consistently requiring about 0.01s - 0.14s initially for parsing, its solving speed remains exceptionally stable across all difficulty levels.

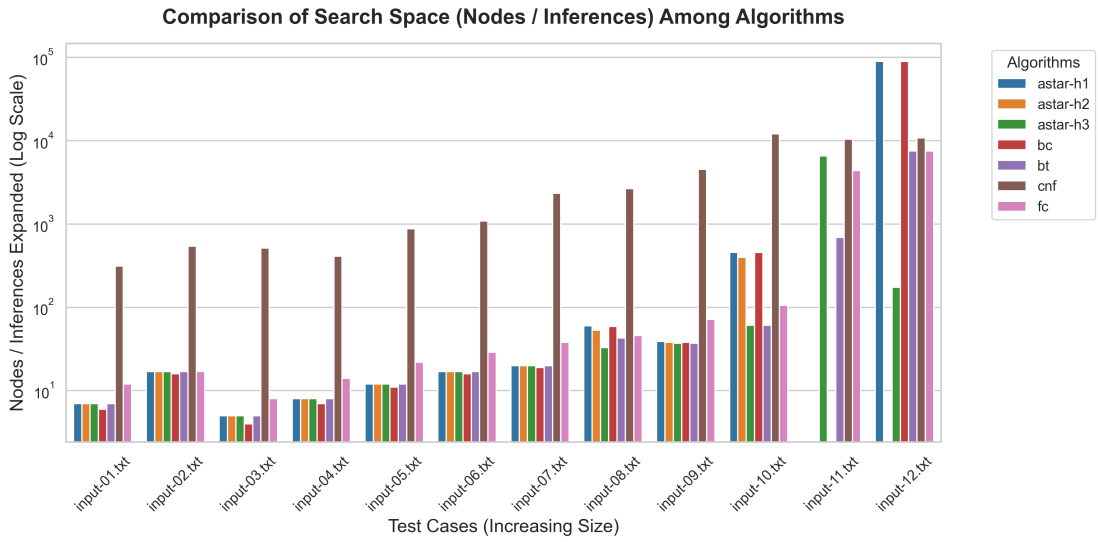



Figure 7: Comparison of Search Space (Nodes / Inferences Expanded).

**Search Space Evaluation (Figure 7):** The vertical axis represents the "Workload" (`work_done = nodes + inferences`). The chart perfectly illustrates the academic power of Constraint algorithms.

- Traditional traversal methods (BT) or those with weak pruning ($A^* - h_1$, BC) exhibit a violent explosion in the search space. Specifically, for `input-12`, $A^*(h_1)$ had to expand nearly **89,566 Nodes** to reach the goal.
- Conversely, the $A^*$ **combined with AC-3** ($h_3$) algorithm features a graph line that practically hugs the baseline. For the same `input-12`, it only required the expansion of a mere **174 Nodes** (almost a straight path with zero backtracking). This serves as absolute mathematical proof of AC-3's exhaustive pruning capabilities. The SAT algorithm also shows a relatively high Inferences metric (reaching over 12,000 in `input-10`) due to the clause traversal mechanism inherent in SAT Solvers.

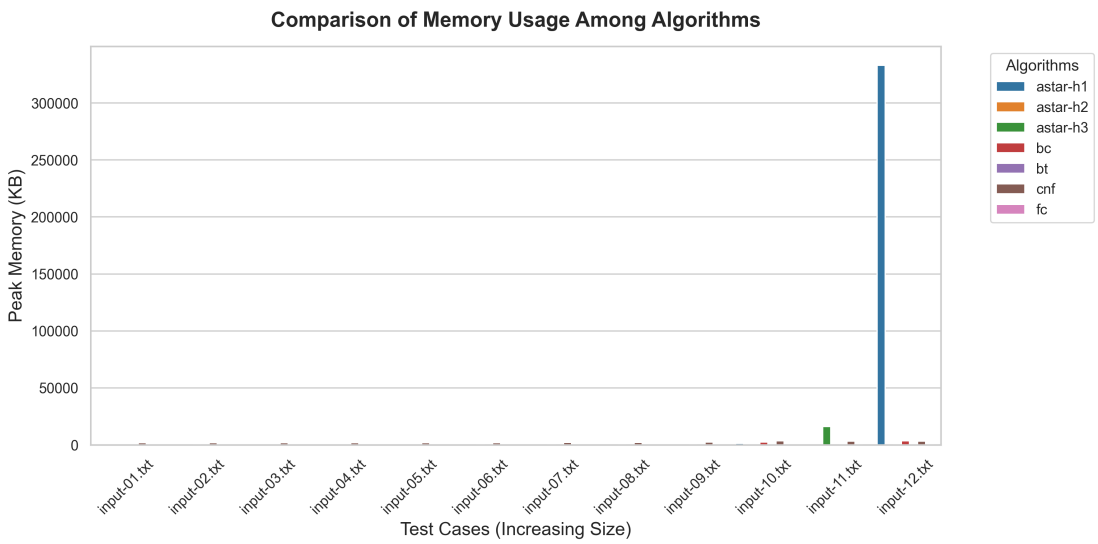



Figure 8: Comparison of Peak Memory Usage.

**Memory Usage Evaluation (Figure 8):** Observing the memory chart provides highly valuable insights into algorithmic architectures:

- **SAT (CNF) Method:** Consumes a large and stable baseline amount of RAM (fluctuating between 1,800 KB and 3,500 KB). This is mandatory as the system must generate and store thousands of DIMACS logic clauses in memory to encode the game rules.
- **Recursive Group (FC, BT):** Champions of RAM optimization. Even on difficult inputs, memory consumption typically hovers around a few tens to hundreds of KB due to the nature of DFS, which only stores the Call Stack.
- **The Peril of $A^*$ (Priority Queue):** Because $A^*$ must store all frontier states in a Priority Queue (Heap) structure, its memory is directly proportional to the number of generated Nodes. For `input-12`, because Heuristic $h_1$ is excessively weak, the queue held tens of thousands of states, causing RAM usage to spike to **333,063 KB (approx. 333 MB)**. Fortunately, thanks to the intelligence of $h_3$ (keeping the Node count consistently low), the memory footprint of $A^*(h_3)$ was controlled at a highly secure level (only 570 KB).

# 7 Comparative Analysis

This section compares the algorithms we implemented, drawing on the benchmark data from Section 7 to answer three questions: which algorithm works best, why, and when you should pick one over another.

## 7.1 Runtime Comparison

Table 12 shows how each solver scales as puzzles get harder. The columns go from a simple $4 \times 4$ grid to the two hardest $9 \times 9$ instances in our test suite.

| **Algorithm** | **input-01** (4×4) | **input-07** (6×6) | **input-10** (9×9) | **input-12** (9×9, sparse) |
|---|---|---|---|---|
| FC + BT | 0.0001 s | 0.0003 s | 0.0017 s | 0.2523 s |
| BT | 0.0003 s | 0.0007 s | 0.0025 s | 0.2618 s |
| BC | 0.0006 s | 0.0014 s | 0.0525 s | 9.3182 s |
| $A^*$ ($h_1$) | 0.0001 s | 0.0008 s | 0.0483 s | 7.5115 s |
| $A^*$ ($h_2$) | 0.0001 s | 0.0017 s | 0.1276 s | *Timeout* |
| $A^*$ ($h_3$) | 0.0002 s | 0.0037 s | 0.1740 s | 0.4572 s |
| SAT (CNF) | 0.1314 s | 0.0232 s | 0.0214 s | 0.0423 s |

Table 12: Running time across increasing puzzle difficulty.

A few things jump out from this table. On the easy puzzles (input-01 through input-07), BT and FC are the clear winners—they finish in fractions of a millisecond because there is almost no search involved. SAT is actually the slowest on the trivial $4 \times 4$ case (0.13 s) because it has to build the entire clause database before solving even begins.

But the picture flips on hard puzzles. On input-12, SAT solves it in 0.04 s as if nothing changed, while BT needs 0.26 s and BC takes over 9 seconds. $A^*(h_2)$ does not finish at all. The only A* variant that survives is $h_3$ (with AC-3), which takes 0.46 s—slower than SAT and BT, but still reasonable.

## 7.2 Memory and Search Effort

Table 13 shows how much memory each algorithm uses and how many nodes it explores on the hardest benchmark.

| Algorithm | Memory (KB) | Nodes | Why |
|---|---|---|---|
| BT | 343 | 7,544 | Only stores the current path (call stack) |
| FC + BT | 359 | 7,544 | Same as BT; FC adds a small domain cache |
| BC | 3,538 | 89,565 | Copies entire state dictionary per branch |
| $A^*$ ($h_1$) | 333,063 | 89,566 | Keeps all generated states in a heap |
| $A^*$ ($h_3$) | 570 | 174 | AC-3 prunes the heap down to almost nothing |
| SAT | 3,372 | 10,775 | Stores the CNF clause database |

Table 13: Memory and node count on input-12.txt ($9 \times 9$, sparse).

The standout number here is $A^*(h_1)$: 333 MB of memory just to solve a $9 \times 9$ puzzle. This happens because $h_1$ cannot tell good states from bad ones, so the priority queue fills up with tens of thousands of partial assignments. $A^*(h_3)$, by contrast, uses AC-3 to figure out early that most branches are hopeless, so only 174 states ever enter the queue and memory stays under 600 KB.

BT and FC are at the other extreme—under 400 KB in all cases, because depth-first search only keeps the current path in memory.

## 7.3 Why Each Algorithm Behaves the Way It Does

**SAT** is the most stable solver because the Glucose4 engine uses Conflict-Driven Clause Learning (CDCL). When it hits a dead-end, it does not just backtrack—it figures out *which* combination of variable assignments caused the contradiction, adds a new clause to prevent that combination from ever being tried again, and then jumps back to the decision point that matters. This means SAT never wastes time re-exploring the same failed pattern. The cost is that the full puzzle must be encoded as CNF clauses upfront, which uses about 2–3 MB regardless of difficulty.

**BT with FC-1** is fast because each node is cheap: one call to `compute_domain` for each affected cell, no heap operations, no clause bookkeeping. Combined with MRV (always pick the cell with fewest remaining values), the search almost always goes straight to the answer. On 11 out of 12 puzzles, BT achieves zero backtracks. The weakness is that FC-1 only checks one step ahead. If cell $A$ constrains $B$ which constrains $C$, assigning $A$ updates $B$'s domain but does not touch $C$ until $B$ is assigned later. This means BT occasionally misses an early dead-end that a smarter propagation method would catch.

$A^*(h_3)$ spends 5 to 50 times more computation per node than BT, because it runs AC-3 arc consistency on every candidate state. But each AC-3 call often shrinks all remaining domains to size 1, which means the search tree collapses to a straight line. On input-12, this trade-off pays off dramatically: 174 nodes expanded versus BT's 7,544. On easier puzzles, though, the overhead is not worth it—BT already reaches the solution in microseconds without needing arc consistency.

**BC** performs poorly compared to BT, even though both do the same depth-first search at heart. The reason is implementation overhead. BC wraps the search inside a Prolog-like SLD-resolution engine: every recursive call involves dictionary copying for state management, variable substitution, occurs-check during unification, and standardize-apart for rule reuse. These

operations add up. On input-12, BC and $A^*(h_1)$ expand nearly the same number of nodes (89,565 vs. 89,566), but BC takes 9.3 s compared to 7.5 s for $A^*(h_1)$ because of this extra per-node cost. BC also lacks memoization, so it may re-solve identical subproblems.

## 7.4 Does Forward Chaining Actually Help Backtracking?

Since FC always falls back to BT when it gets stuck, the total cost is the FC propagation phase plus BT on whatever is left. Whether this is faster than running BT alone depends on how much FC can derive.

| **Input** | **FC derived** | **BT still needed** | **FC+BT time** | **BT-only time** |
|---|---|---|---|---|
| input-09 (dense) | 36 / 36 cells | 0 nodes | 0.0005 s | 0.0016 s |
| input-10 | 45 / 61 cells | 16 nodes | 0.0017 s | 0.0025 s |
| input-11 (sparse) | 1 / 61 cells | 4,373 nodes | 0.1796 s | 0.0231 s |
| input-12 (sparse) | 2 / 61 cells | 7,542 nodes | 0.2523 s | 0.2618 s |

Table 14: When FC helps and when it does not.

On input-09, FC solves the entire puzzle by itself because the constraints are tight enough that every empty cell eventually gets forced to a single value. BT is never called, and FC+BT finishes 3× faster than BT alone.

On input-10, FC fills 45 out of 61 cells, then BT handles the remaining 16. This is still faster overall because BT starts from a much-reduced search space.

On input-11, FC can only derive 1 fact. The FC propagation phase costs time but produces almost nothing, and then BT has to do all the real work anyway. Here, BT alone is about 8× faster.

The pattern is simple: **FC helps when the puzzle has many inequality constraints**, because each inequality propagation can trigger a chain reaction that forces values across the board. On sparse puzzles where there are few constraints to propagate, FC wastes time scanning rows, columns, and inequality chains without finding anything useful.

## 7.5 When to Use Which Algorithm

| Situation | Use this | Why |
|---|---|---|
| You need a solver that never times out | SAT (CNF) | CDCL clause learning avoids combinatorial explosion. Runtime stays between 0.02–0.14 s on all our benchmarks, regardless of difficulty. |
| Small or medium puzzles, speed matters most | BT (or FC+BT if constraints are dense) | Lowest overhead per step. Finishes most puzzles in under 3 ms. |
| Memory is very limited | BT | DFS uses under 500 KB. No heap, no clause database. |
| You want the fewest wrong guesses | $A^*$ + AC-3 | AC-3 detects dead-ends before exploring them. Expanded only 174 nodes on the hardest puzzle. |
| You need interactive queries on individual cells | BC | The Prolog-style REPL lets you query `Val(R,C,?X)` directly. No other solver supports this. |

Table 15: Practical guide for choosing an algorithm.

## 7.6 Summary

No single algorithm is the best at everything. SAT is the safest choice when reliability matters—it solved every puzzle we threw at it without breaking a sweat, thanks to CDCL. BT is the fastest choice for everyday puzzles and uses almost no memory. $A^*(h_3)$ is the smartest searcher, finding solutions with minimal exploration, but it pays for that intelligence with higher per-node cost. FC is a useful add-on to BT when constraints are plentiful, but counterproductive when they are sparse. BC fills a niche for interactive logic queries but is too slow for serious solving.

The common thread across all our experiments is a trade-off between **how hard you think per step** and **how many steps you take**. Algorithms that invest more upfront reasoning (SAT with clause learning, $A^*$ with AC-3) end up exploring far fewer states. Algorithms that keep each step cheap (BT, FC) are faster on easy problems but can struggle when the search tree gets large. Understanding this trade-off is the key to picking the right tool for the job.

# 8 References

The following references summarize the academic and technical foundations most relevant to the concepts, algorithms, and modeling choices used throughout this report.

[1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach.* 4th ed., Pearson, 2021.

[2] Ivan Bratko. *Prolog Programming for Artificial Intelligence.* 4th ed., Addison-Wesley, 2012.

[3] Rina Dechter. *Constraint Processing.* Morgan Kaufmann, 2003.

[4] Richard E. Korf. "Depth-first iterative-deepening: An optimal admissible tree search." *Artificial Intelligence*, vol. 27, no. 1, 1985, pp. 97–109.

[5] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, 1968, pp. 100–107.

[6] Alan K. Mackworth. "Consistency in networks of relations." *Artificial Intelligence*, vol. 8, no. 1, 1977, pp. 99–118.

[7] Judea Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving.* Addison-Wesley, 1984.

[8] Martin Davis, George Logemann, and Donald Loveland. "A machine program for theorem-proving." *Communications of the ACM*, vol. 5, no. 7, 1962, pp. 394–397.

[9] Joao Marques-Silva and Karem A. Sakallah. "GRASP: A Search Algorithm for Propositional Satisfiability." *IEEE Transactions on Computers*, vol. 48, no. 5, 1999, pp. 506–521.

[10] Niklas Een and Niklas Sorensson. "An Extensible SAT-solver." *Theory and Applications of Satisfiability Testing (SAT 2003)*, LNCS 2919, Springer, 2004, pp. 502–518.

[11] Robert Kowalski. *Logic for Problem Solving.* Elsevier North-Holland, 1979.

[12] Francesca Rossi, Peter van Beek, and Toby Walsh, editors. *Handbook of Constraint Programming.* Elsevier, 2006.

[13] Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability.* Addison-Wesley, 2015.

[14] Futoshiki puzzle rules and examples were adapted from standard puzzle descriptions used in logic-puzzle references and online puzzle collections; the formalization, solver implementations, experiments, and comparative analysis in this project are the authors' own work.